Introduction to Recurrent Neural Networks (RNNs)

Balaji Srinivasan, Ganapathy Krishnamurthi

Wadhwani School of AI , IIT Madras

Outline of this Module

1. Introduction to Sequence Modelling
2. Introduction to Recurrent Neural Networks
3. Advanced Memory based Models
4. Modern Recurrent Neural Networks

The Sequential Data

Introduction to Sequence Modelling

Introduction to Sequential data

What is Sequential Data?

- Sequential data is a type of data where the order of the data points is significant. The sequence itself contains important information.

Introduction to Sequential data

What is Sequential Data?

- Sequential data is a type of data where the order of the data points is significant. The sequence itself contains important information.
- Each data point is dependent on the previous ones (e.g., a sentence). This creates temporal or spatial dependencies within the data.

Introduction to Sequential data

What is Sequential Data?

- Sequential data is a type of data where the order of the data points is significant. The sequence itself contains important information.
- Each data point is dependent on the previous ones (e.g., a sentence). This creates temporal or spatial dependencies within the data.

Key Characteristics

Introduction to Sequential data

What is Sequential Data?

- Sequential data is a type of data where the order of the data points is significant. The sequence itself contains important information.
- Each data point is dependent on the previous ones (e.g., a sentence). This creates temporal or spatial dependencies within the data.

Key Characteristics

- **Ordered**: Data points have a specific position or index in a sequence.

Introduction to Sequential data

What is Sequential Data?

- Sequential data is a type of data where the order of the data points is significant. The sequence itself contains important information.
- Each data point is dependent on the previous ones (e.g., a sentence). This creates temporal or spatial dependencies within the data.

Key Characteristics

- **Ordered**: Data points have a specific position or index in a sequence.
- **Variable Length**: Sequences can have different lengths (e.g., different sentence lengths).

Introduction to Sequential data

What is Sequential Data?

- Sequential data is a type of data where the order of the data points is significant. The sequence itself contains important information.
- Each data point is dependent on the previous ones (e.g., a sentence). This creates temporal or spatial dependencies within the data.

Key Characteristics

- **Ordered**: Data points have a specific position or index in a sequence.
- **Variable Length**: Sequences can have different lengths (e.g., different sentence lengths).
- **Dependencies**: Values can depend on other values at different positions in the sequence.

Introduction to Sequential data

What is Sequential Data?

- Sequential data is a type of data where the order of the data points is significant. The sequence itself contains important information.
- Each data point is dependent on the previous ones (e.g., a sentence). This creates temporal or spatial dependencies within the data.

Key Characteristics

- **Ordered**: Data points have a specific position or index in a sequence.
- **Variable Length**: Sequences can have different lengths (e.g., different sentence lengths).
- **Dependencies**: Values can depend on other values at different positions in the sequence.

Key Concept

Analyzing this data involves understanding patterns, trends, and contexts **over time or space**.

Common Sequential data and Tasks

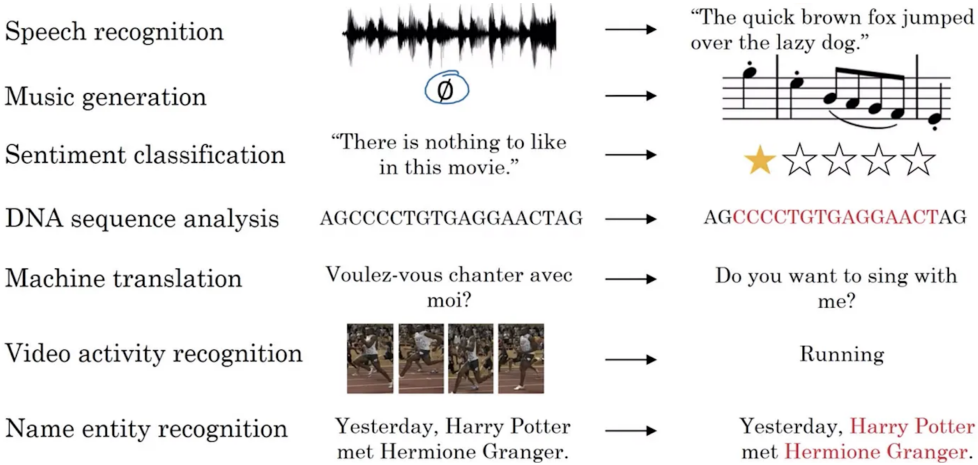


Figure: Common Sequential data and Tasks. (source: AIML.com)

Sequence Models

Mathematical Formulation

- A sequence model aims to estimate the joint probability of an entire sequence of observations.

Sequence Models

Mathematical Formulation

- A sequence model aims to estimate the joint probability of an entire sequence of observations.
- For a sequence $x_1, x_2, \ldots, x_T$, the model estimates:

$$P(x_1, x_2, \ldots, x_T)$$

Sequence Models

Mathematical Formulation

- A sequence model aims to estimate the joint probability of an entire sequence of observations.
- For a sequence $x_1, x_2, \ldots, x_T$, the model estimates:

$$P(x_1, x_2, \ldots, x_T)$$

- This allows us to assess how plausible a sequence is, or to generate new, similar sequences.

Sequence Models

Mathematical Formulation

- A sequence model aims to estimate the joint probability of an entire sequence of observations.
- For a sequence $x_1, x_2, \ldots, x_T$, the model estimates:

$$P(x_1, x_2, \ldots, x_T)$$

- This allows us to assess how plausible a sequence is, or to generate new, similar sequences.

Language Models

When a sequence model is applied to text data, it is called a **language model**. Its goal is to estimate the probability of a sequence of words or characters.

Autoregressive Models

Definition

- Autoregressive models predict future values in a sequence based on past values.

Autoregressive Models

Definition

- Autoregressive models predict future values in a sequence based on past values.
- This is achieved by factoring the joint probability of the sequence using the chain rule of probability:

$$P(x_1, \ldots, x_T) = P(x_1) \prod_{t=2}^{T} P(x_t \mid x_1, \ldots, x_{t-1})$$

- In words: the probability of a sequence is the product of the conditional probabilities of each element, given the elements that came before it.

Autoregressive Models

Definition

- Autoregressive models predict future values in a sequence based on past values.
- This is achieved by factoring the joint probability of the sequence using the chain rule of probability:

$$P(x_1, \ldots, x_T) = P(x_1) \prod_{t=2}^{T} P(x_t \mid x_1, \ldots, x_{t-1})$$

- In words: the probability of a sequence is the product of the conditional probabilities of each element, given the elements that came before it.

How it Works

To predict the next element x_t, the model uses the preceding sequence $(x_1, \ldots, x_{t-1})$ as input.

The Markov Assumption

- Handling very long sequences can be computationally expensive because the model would need to consider the entire history.
- To simplify this, we can use the **Markov assumption**.

The Markov Assumption

- Handling very long sequences can be computationally expensive because the model would need to consider the entire history.
- To simplify this, we can use the **Markov assumption**.

First-Order Markov Model

- Assumes that the current state x_t only depends on the immediately preceding state x_{t-1}.
- The probability formula simplifies to:

$$P(x_1, \ldots, x_T) \approx P(x_1) \prod_{t=2}^{T} P(x_t \mid x_{t-1})$$

- This is also known as a bigram model in the context of language processing.

The Markov Assumption

- Handling very long sequences can be computationally expensive because the model would need to consider the entire history.
- To simplify this, we can use the **Markov assumption**.

First-Order Markov Model

- Assumes that the current state x_t only depends on the immediately preceding state x_{t-1}.
- The probability formula simplifies to:

$$P(x_1, \ldots, x_T) \approx P(x_1) \prod_{t=2}^{T} P(x_t \mid x_{t-1})$$

- This is also known as a bigram model in the context of language processing.

- This can be extended to consider a fixed-length window of previous states (e.g., trigrams, etc.).

Linear Regression for Sequence modelling

- It follows an n-gram Markovian assumption
- It uses a fixed window of past data and applies a linear model to predict the next value.

Linear Regression for Sequence modelling

- It follows an n-gram Markovian assumption
- It uses a fixed window of past data and applies a linear model to predict the next value.

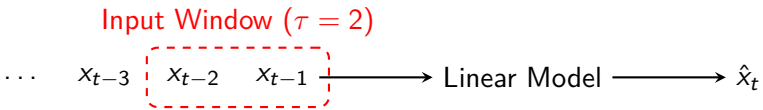


Diagram showing a linear model predicting x_t using a window of the two previous time steps.

Linear Regression for Sequence modelling

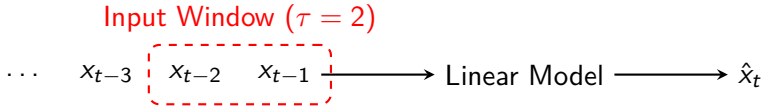


Diagram showing a linear model predicting x_t using a window of the two previous time steps.

Mathematical Formulation

The prediction, $\hat{x}_t$, is a weighted sum of the previous τ observations plus a bias term:

$$\hat{x}_t = \sum_{i=1}^{\tau} w_i x_{t-i} + b$$

- x_{t-i}: Past values (our features).
- w_i: Model weights to be learned.
- b: A bias term.

Limitations of Linear Regression for Sequence modelling

1. The Linearity Assumption

Linear models assume that the future is just a simple weighted sum of the past. They fail to capture complex, non-linear relationships, such as:

- The interaction between different past events.
- Sudden shocks or spikes in the data (e.g., a stock market crash).

Limitations of Linear Regression for Sequence modelling

1. The Linearity Assumption

Linear models assume that the future is just a simple weighted sum of the past. They fail to capture complex, non-linear relationships, such as:

- The interaction between different past events.
- Sudden shocks or spikes in the data (e.g., a stock market crash).

2. Lack of Long-Term Memory (Fixed Window)

By conditioning only on a fixed-length window (τ), the model is completely blind to any information that occurred before that window. This is a major drawback when long-range dependencies are important.

Limitations of Linear Regression for Sequence modelling

3. Error Accumulation

For k-step-ahead predictions (predicting $\hat{x}_{t+k}$), the model must use its own previous predictions as input. This creates a feedback loop where errors compound over time.

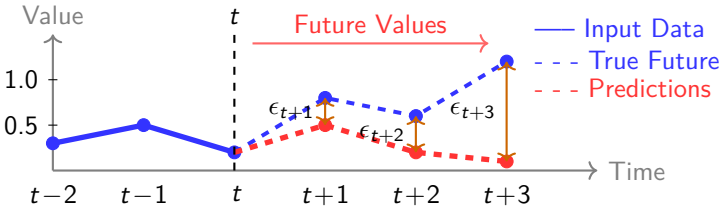
- The prediction $\hat{x}_{t+1}$ will have some error, ϵ_1.
- This incorrect value, $\hat{x}_{t+1}$, is then fed back into the model to predict $\hat{x}_{t+2}$.
- The error in the input leads to a new, often larger, error ϵ_2. This process repeats, causing the predictions to **diverge rapidly** from the true data.

Limitations of Linear Regression for Sequence modelling

3. Error Accumulation

For k-step-ahead predictions (predicting $\hat{x}_{t+k}$), the model must use its own previous predictions as input. This creates a feedback loop where errors compound over time.

- The prediction $\hat{x}_{t+1}$ will have some error, ϵ_1.
- This incorrect value, $\hat{x}_{t+1}$, is then fed back into the model to predict $\hat{x}_{t+2}$.
- The error in the input leads to a new, often larger, error ϵ_2. This process repeats, causing the predictions to **diverge rapidly** from the true data.



Limitations of MLPs and CNNs for Sequence Tasks

Why MLPs and CNNs are Not Ideal for Sequences

- **Fixed-Size Vectors:** They require inputs of a fixed length. This forces us to either truncate long sequences or pad short ones, both of which can lead to loss of information.

Limitations of MLPs and CNNs for Sequence Tasks

Why MLPs and CNNs are Not Ideal for Sequences

- **Fixed-Size Vectors:** They require inputs of a fixed length. This forces us to either truncate long sequences or pad short ones, both of which can lead to loss of information.
- **No Parameter Sharing Across Time:** An MLP learns separate weights for features at each position. It cannot generalize a pattern it learns at position ‘i‘ to position ‘j‘. A CNN shares parameters spatially, but isn’t designed for capturing temporal dependencies of arbitrary length.

Limitations of MLPs and CNNs for Sequence Tasks

Why MLPs and CNNs are Not Ideal for Sequences

- **Fixed-Size Vectors:** They require inputs of a fixed length. This forces us to either truncate long sequences or pad short ones, both of which can lead to loss of information.
- **No Parameter Sharing Across Time:** An MLP learns separate weights for features at each position. It cannot generalize a pattern it learns at position ‘i‘ to position ‘j‘. A CNN shares parameters spatially, but isn’t designed for capturing temporal dependencies of arbitrary length.
- **No Memory of Context:** They process each part of the input independently. They have no mechanism to remember previous elements when processing a new one.

Limitations of MLPs and CNNs for Sequence Tasks

Why MLPs and CNNs are Not Ideal for Sequences

- **Fixed-Size Vectors:** They require inputs of a fixed length. This forces us to either truncate long sequences or pad short ones, both of which can lead to loss of information.
- **No Parameter Sharing Across Time:** An MLP learns separate weights for features at each position. It cannot generalize a pattern it learns at position ‘i‘ to position ‘j‘. A CNN shares parameters spatially, but isn't designed for capturing temporal dependencies of arbitrary length.
- **No Memory of Context:** They process each part of the input independently. They have no mechanism to remember previous elements when processing a new one.

We need a model that processes sequences step-by-step, with memory.

Language modelling

Text preprocessing tricks

From Text to Sequences

The Goal: Making Text Understandable to Machines

Neural networks don't understand words like "hello" or "world". They only understand numbers. Our first and most critical task is to convert our raw text corpus into a numerical format that a model can process.

From Text to Sequences

The Goal: Making Text Understandable to Machines

Neural networks don't understand words like "hello" or "world". They only understand numbers. Our first and most critical task is to convert our raw text corpus into a numerical format that a model can process.

Raw Text	→	Tokens	→	Numerical Sequence
"The cat sat..."		["the", "cat", "sat", ...]		[1, 2, 3, ...]
Step 1		**Step 2**		**Step 3**
Read Data		Tokenize		Create Vocabulary

From Text to Sequences

The Goal: Making Text Understandable to Machines

Neural networks don't understand words like "hello" or "world". They only understand numbers. Our first and most critical task is to convert our raw text corpus into a numerical format that a model can process.

Raw Text	→	Tokens	→	Numerical Sequence
"The cat sat..."		["the", "cat", "sat", ...]		[1, 2, 3, ...]
Step 1 Read Data		**Step 2** Tokenize		**Step 3** Create Vocabulary

The Core Idea

We will build a dictionary (a **vocabulary**) that maps every unique word or character to a unique integer index.

Step 1 & 2: Tokenization and Building a Vocabulary

Tokenization: Breaking Text into Pieces

The first step is to split the text into smaller units called **tokens**. We have two main choices:

- **Character Tokens:** Split the text into individual characters ('h', 'e', 'l', 'l', 'o'). This results in a small vocabulary but very long sequences.
- **Word Tokens:** Split the text by spaces and punctuation ('hello', 'world', '.'). This is more common and results in a larger vocabulary but shorter, more meaningful sequences.

Step 1 & 2: Tokenization and Building a Vocabulary

Tokenization: Breaking Text into Pieces

The first step is to split the text into smaller units called **tokens**. We have two main choices:

- **Character Tokens:** Split the text into individual characters ('h', 'e', 'l', 'l', 'o'). This results in a small vocabulary but very long sequences.
- **Word Tokens:** Split the text by spaces and punctuation ('hello', 'world', '.'). This is more common and results in a larger vocabulary but shorter, more meaningful sequences.

Building the Vocabulary

Find all unique tokens in the text corpus and then assign a unique integer index to each one.

- **Corpus:** "The cat sat on the wall."
- **Unique Word Tokens:** {"the", "cat", "sat", "on", "wall"}
- **Vocabulary (Vocab):** We create a mapping. It's also crucial to add a special token for any words that might appear later but are not in our current vocabulary.

Step 3: Creating the Final Numerical Sequence

Mapping Tokens to Indices

With our vocabulary complete, we can now convert any sentence into a sequence of integers. We simply replace each token with its corresponding index from the vocabulary.

Step 3: Creating the Final Numerical Sequence

Mapping Tokens to Indices

With our vocabulary complete, we can now convert any sentence into a sequence of integers. We simply replace each token with its corresponding index from the vocabulary.

Example Conversion

- **Vocabulary:** `{"<unk>": 0, "the": 1, "cat": 2, "sat": 3, "on": 4, "wall": 5}`

Step 3: Creating the Final Numerical Sequence

Mapping Tokens to Indices

With our vocabulary complete, we can now convert any sentence into a sequence of integers. We simply replace each token with its corresponding index from the vocabulary.

Example Conversion

- **Vocabulary:** `{"<unk>": 0, "the": 1, "cat": 2, "sat": 3, "on": 4, "wall": 5}`
- **Sentence 1:** "the cat sat on the wall"
- **Numerical Sequence:** `[1, 2, 3, 4, 1, 5]`

Step 3: Creating the Final Numerical Sequence

Mapping Tokens to Indices

With our vocabulary complete, we can now convert any sentence into a sequence of integers. We simply replace each token with its corresponding index from the vocabulary.

Example Conversion

- **Vocabulary:** `{"<unk>": 0, "the": 1, "cat": 2, "sat": 3, "on": 4, "wall": 5}`
- **Sentence 1:** "the cat sat on the wall"
- **Numerical Sequence:** `[1, 2, 3, 4, 1, 5]`
- **Sentence 2:** "the cat sat on the chair"
- *Note: "chair" is not in our vocabulary!*
- **Numerical Sequence:** `[1, 2, 3, 4, 1, 0]` (*Here, "chair" is mapped to the <unk> token index, 0*)

Mathematics of Language Modelling

The Goal: Estimating Sequence Probability

A language model aims to answer a fundamental question: What is the probability of a given sequence of words?

Mathematics of Language Modelling

The Goal: Estimating Sequence Probability

A language model aims to answer a fundamental question: What is the probability of a given sequence of words?

For a sequence $x_1, x_2, \ldots, x_T$, the model estimates the joint probability: $P(x_1, x_2, \ldots, x_T)$

Mathematics of Language Modelling

The Goal: Estimating Sequence Probability

A language model aims to answer a fundamental question: What is the probability of a given sequence of words?

For a sequence $x_1, x_2, \ldots, x_T$, the model estimates the joint probability: $P(x_1, x_2, \ldots, x_T)$

The Chain Rule of Probability

We can factor this joint probability into a product of conditional probabilities. This is the core of modern autoregressive language models.

$$P(x_1, \ldots, x_T) = P(x_1) \prod_{t=2}^{T} P(x_t \mid x_1, \ldots, x_{t-1})$$

- The probability of a word depends on all the words that came before it.

Mathematics of Language Modelling

The Chain Rule of Probability

We can factor this joint probability into a product of conditional probabilities. This is the core of modern autoregressive language models.

$$P(x_1, \ldots, x_T) = P(x_1) \prod_{t=2}^{T} P(x_t \mid x_1, \ldots, x_{t-1})$$

- The probability of a word depends on all the words that came before it.

The Challenge

The conditioning term $P(x_t \mid x_1, \ldots, x_{t-1})$ becomes computationally expensive and statistically difficult to model as the sequence length t grows.

Partitioning Sequences

Preparing Data for Neural Models

To train a neural language model, we need to create feature-label pairs from our long text sequence. There are two common strategies – Random Sampling and Sequential Partitioning.

Partitioning Sequences

Preparing Data for Neural Models

To train a neural language model, we need to create feature-label pairs from our long text sequence. There are two common strategies – Random Sampling and Sequential Partitioning.

Random Sampling

- To introduce randomness, a random number of tokens is discarded from the start of the corpus at the beginning of each epoch.
- The rest of the corpus is then partitioned into multiple non-overlapping subsequences of a fixed length, τ.
- For any given input subsequence $X = (x_1, x_2, \ldots, x_\tau)$, the target (label) sequence Y is the same sequence shifted by one token to the left: $Y = (x_2, x_3, \ldots, x_{\tau+1})$.
- Each subsequence created this way is treated as an independent example for training.

Partitioning Sequences

Original Corpus:

The | cat | sat | on | the | mat | and | slept | all | day

Partitioning Sequences

Original Corpus:

The | cat | sat | on | the | mat | and | slept | all | day

Random Discard:

~~The~~ | ~~cat~~ | sat | on | the | mat | and | slept | all | day

Partitioning Sequences

Original Corpus:

The	cat	sat	on	the	mat	and	slept	all	day

Random Discard:

~~The~~	~~cat~~	sat	on	the	mat	and	slept	all	day

Partition ($\tau = 3$):

sat	on	the	mat	and	slept	···	

Partitioning Sequences

Original Corpus:

The | cat | sat | on | the | mat | and | slept | all | day

Random Discard:

~~The~~ | ~~cat~~ | sat | on | the | mat | and | slept | all | day

Partition ($\tau = 3$):

sat | on | the | mat | and | slept | ⋯

Input-Target Pairs:

Input1: sat | on | the

Target1: on | the | mat

$X = (x_1, x_2, x_3)$
$Y = (x_2, x_3, x_4)$

Input2: mat | and | slept

Target2: and | slept | all

$X = (x_4, x_5, x_6)$
$Y = (x_5, x_6, x_7)$

Each subsequence = Independent training example

Perplexity

How Good Is a Language Model?

Perplexity (PPL) is the standard metric for evaluating language models. It measures how "surprised" or "perplexed" a model is by a sequence of text.

Lower Perplexity is Better!

Perplexity

How Good Is a Language Model?

Perplexity (PPL) is the standard metric for evaluating language models. It measures how "surprised" or "perplexed" a model is by a sequence of text.

Lower Perplexity is Better!

Mathematical Definition

Perplexity is the exponentiation of the average negative log-likelihood of the sequence. For a sequence of length T:

$$\text{PPL} = \exp\left(-\frac{1}{T}\sum_{t=1}^{T}\log P(x_t \mid x_1, \ldots, x_{t-1})\right)$$

Perplexity

Intuition

A perplexity of k means that, on average, the model is as uncertain about the next word as if it had to choose uniformly from k different words.

- If PPL $= 100$, the model is as confused as someone randomly guessing from 100 words.
- If PPL $= 10$, the model has narrowed down the possibilities to about 10 words.

Recurrent Neural Networks

Memory based models for sequential data

Introduction to Recurrent Neural Networks (RNNs)

Markov models and n-grams have a **fixed memory**; they only look at the last $n - 1$ tokens.

Introduction to Recurrent Neural Networks (RNNs)

Markov models and n-grams have a **fixed memory**; they only look at the last $n-1$ tokens. **To remember more, the number of model parameters grows exponentially**, making them impractical for capturing long-range dependencies.

Introduction to Recurrent Neural Networks (RNNs)

Markov models and n-grams have a **fixed memory**; they only look at the last $n-1$ tokens. **To remember more, the number of model parameters grows exponentially**, making them impractical for capturing long-range dependencies.

The RNN Solution: A Latent Hidden State

Instead of conditioning on the raw sequence of past tokens, an RNN maintains a compact summary of the entire history in a **hidden state**, h_t.

- The model approximates the true probability with the hidden state:

$$P(x_t \mid x_1, \ldots, x_{t-1}) \approx P(x_t \mid h_{t-1})$$

- This state is updated at each time step using the current input x_t and the previous state h_{t-1}:

$$h_t = f(x_t, h_{t-1})$$

The Recurrent Neural Network

The Core Idea: A Loop with Memory

- An RNN processes a sequence by iterating through its elements one by one.

The Recurrent Neural Network

The Core Idea: A Loop with Memory

- An RNN processes a sequence by iterating through its elements one by one.
- It maintains a **hidden state** (or "memory") that captures information about what it has seen so far.

The Recurrent Neural Network

The Core Idea: A Loop with Memory

- An RNN processes a sequence by iterating through its elements one by one.
- It maintains a **hidden state** (or "memory") that captures information about what it has seen so far.
- The output at a given time step depends not only on the current input but also on the hidden state from the previous time step.

The Recurrent Neural Network

The Core Idea: A Loop with Memory

- An RNN processes a sequence by iterating through its elements one by one.
- It maintains a **hidden state** (or "memory") that captures information about what it has seen so far.
- The output at a given time step depends not only on the current input but also on the hidden state from the previous time step.

RNN Basics

- RNNs process sequential data by maintaining a **hidden state** that captures information seen so far

The Recurrent Neural Network

The Core Idea: A Loop with Memory

- An RNN processes a sequence by iterating through its elements one by one.
- It maintains a **hidden state** (or "memory") that captures information about what it has seen so far.
- The output at a given time step depends not only on the current input but also on the hidden state from the previous time step.

RNN Basics

- RNNs process sequential data by maintaining a **hidden state** that captures information seen so far
- At each time step, the network:

The Recurrent Neural Network

The Core Idea: A Loop with Memory

- An RNN processes a sequence by iterating through its elements one by one.
- It maintains a **hidden state** (or "memory") that captures information about what it has seen so far.
- The output at a given time step depends not only on the current input but also on the hidden state from the previous time step.

RNN Basics

- RNNs process sequential data by maintaining a **hidden state** that captures information seen so far
- At each time step, the network:
  - Takes a new input x_t

The Recurrent Neural Network

The Core Idea: A Loop with Memory

- An RNN processes a sequence by iterating through its elements one by one.
- It maintains a **hidden state** (or "memory") that captures information about what it has seen so far.
- The output at a given time step depends not only on the current input but also on the hidden state from the previous time step.

RNN Basics

- RNNs process sequential data by maintaining a **hidden state** that captures information seen so far
- At each time step, the network:
  - Takes a new input x_t
  - Updates its hidden state h_t

The Recurrent Neural Network

The Core Idea: A Loop with Memory

- An RNN processes a sequence by iterating through its elements one by one.
- It maintains a **hidden state** (or "memory") that captures information about what it has seen so far.
- The output at a given time step depends not only on the current input but also on the hidden state from the previous time step.

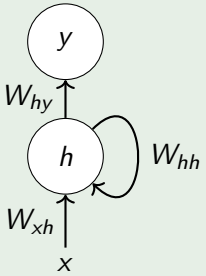
RNN Basics

- RNNs process sequential data by maintaining a **hidden state** that captures information seen so far
- At each time step, the network:
  - Takes a new input x_t
  - Updates its hidden state h_t
  - Produces an output y_t

A Basic Recurrent neuron

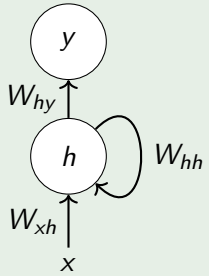
A Basic Recurrent neuron

The Recurrent neuron

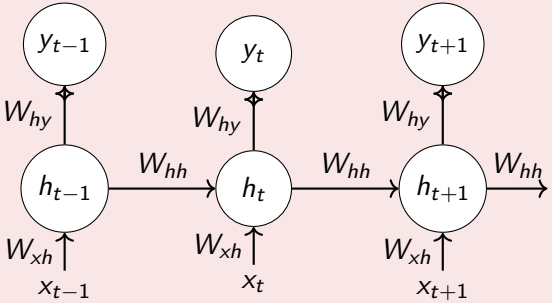


A Basic Recurrent neuron

The Recurrent neuron

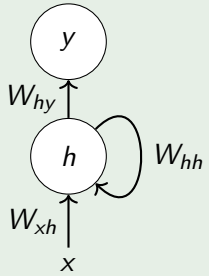


Unfolding Over Time

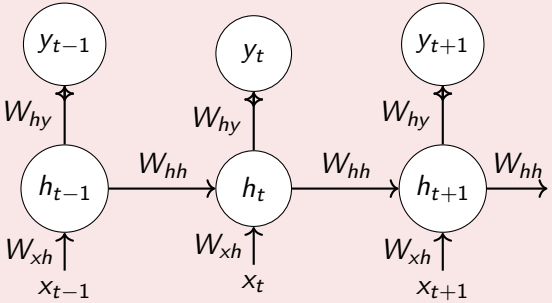


A Basic Recurrent neuron

The Recurrent neuron



Unfolding Over Time

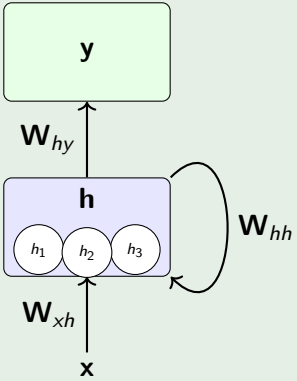


Key Point: The same weight matrix is used at every time step.

A Layer of Recurrent Neurons

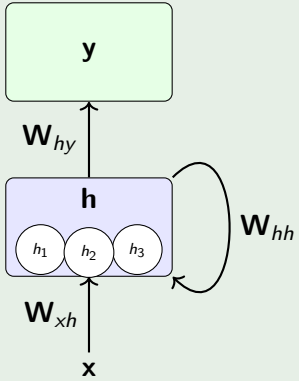
A Layer of Recurrent Neurons

RNN Layer (Folded)

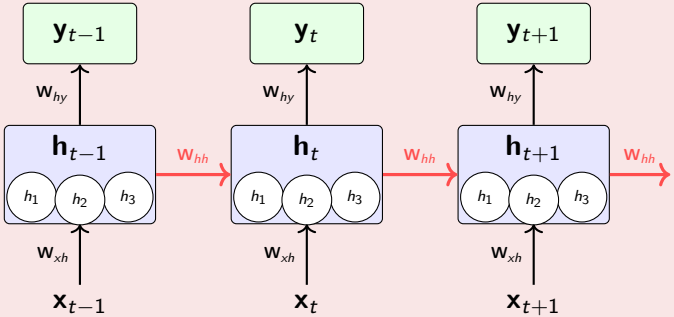


A Layer of Recurrent Neurons

RNN Layer (Folded)



Unfolded RNN Layer Over Time



RNN Computation in a neuron

The Weights of a Single Recurrent Neuron

A single recurrent neuron has three distinct sets of weights that manage its state and output:

RNN Computation in a neuron

The Weights of a Single Recurrent Neuron

A single recurrent neuron has three distinct sets of weights that manage its state and output:

- w_{xh}: Weight for the current input, $x_{(t)}$.

RNN Computation in a neuron

The Weights of a Single Recurrent Neuron

A single recurrent neuron has three distinct sets of weights that manage its state and output:

- w_{xh}: Weight for the current input, $x_{(t)}$.
- w_{hh}: Weight for its own previous hidden state, $h_{(t-1)}$. This is the "recurrent" or "memory" weight.

RNN Computation in a neuron

The Weights of a Single Recurrent Neuron

A single recurrent neuron has three distinct sets of weights that manage its state and output:

- w_{xh}: Weight for the current input, $x_{(t)}$.
- w_{hh}: Weight for its own previous hidden state, $h_{(t-1)}$. This is the "recurrent" or "memory" weight.
- w_{hy}: Weight for producing the final output, $y_{(t)}$, from its current hidden state, $h_{(t)}$.

RNN Computation in a neuron

The Weights of a Single Recurrent Neuron

A single recurrent neuron has three distinct sets of weights that manage its state and output:

- w_{xh}: Weight for the current input, $x_{(t)}$.
- w_{hh}: Weight for its own previous hidden state, $h_{(t-1)}$. This is the "recurrent" or "memory" weight.
- w_{hy}: Weight for producing the final output, $y_{(t)}$, from its current hidden state, $h_{(t)}$.

For one neuron, the computation is:

$$h_{(t)} = \tanh(w_{xh}x_{(t)} + w_{hh}h_{(t-1)} + b_h)$$

and

$$y_{(t)} = w_{hy}h_{(t)} + b_y$$

RNN Computations in a Layer

From Weights to Weight Matrices: The RNN Layer

An RNN layer consists of many neurons. To compute efficiently, we group the individual weights from all neurons into shared weight matrices.

RNN Computations in a Layer

From Weights to Weight Matrices: The RNN Layer

An RNN layer consists of many neurons. To compute efficiently, we group the individual weights from all neurons into shared weight matrices.

- All input weights (w_{xh}) form the matrix $\mathbf{W}_{xh}$.

RNN Computations in a Layer

From Weights to Weight Matrices: The RNN Layer

An RNN layer consists of many neurons. To compute efficiently, we group the individual weights from all neurons into shared weight matrices.

- All input weights (w_{xh}) form the matrix $\mathbf{W}_{xh}$.
- All recurrent weights (w_{hh}) form the matrix $\mathbf{W}_{hh}$.

RNN Computations in a Layer

From Weights to Weight Matrices: The RNN Layer

An RNN layer consists of many neurons. To compute efficiently, we group the individual weights from all neurons into shared weight matrices.

- All input weights (w_{xh}) form the matrix $\mathbf{W}_{xh}$.
- All recurrent weights (w_{hh}) form the matrix $\mathbf{W}_{hh}$.
- All output weights (w_{hy}) form the matrix $\mathbf{W}_{hy}$.

RNN Computations in a Layer

From Weights to Weight Matrices: The RNN Layer

An RNN layer consists of many neurons. To compute efficiently, we group the individual weights from all neurons into shared weight matrices.

- All input weights (w_{xh}) form the matrix $\mathbf{W}_{xh}$.
- All recurrent weights (w_{hh}) form the matrix $\mathbf{W}_{hh}$.
- All output weights (w_{hy}) form the matrix $\mathbf{W}_{hy}$.

This allows us to process an entire vector of inputs $\mathrm{x}_{(t)}$ and update the entire layer's hidden state vector $\mathrm{h}_{(t)}$ at once with the following equations:

$$\mathrm{h}_{(t)} = \tanh(\mathbf{W}_{xh}\mathrm{x}_{(t)} + \mathbf{W}_{hh}\mathrm{h}_{(t-1)} + \mathrm{b}_h)$$
$$\mathrm{y}_{(t)} = \mathbf{W}_{hy}\mathrm{h}_{(t)} + \mathrm{b}_y$$

Numerical Example: Next-Character Prediction

The Task

Given the input character "e" and the memory of having just seen "h", predict the next character. Our simple vocabulary is `{'h':0, 'e':1, 'l':2, 'o':3}`.

Numerical Example: Next-Character Prediction

The Task

Given the input character "e" and the memory of having just seen "h", predict the next character. Our simple vocabulary is `{'h':0, 'e':1, 'l':2, 'o':3}`.

Setup

We use one-hot encoding for characters. The layer has 2 hidden neurons.

Numerical Example: Next-Character Prediction

The Task

Given the input character "e" and the memory of having just seen "h", predict the next character. Our simple vocabulary is `{'h':0, 'e':1, 'l':2, 'o':3}`.

Setup

We use one-hot encoding for characters. The layer has 2 hidden neurons.

- Input $x_{(t)}$ (char 'e'): $\begin{pmatrix} 0 & 1 & 0 & 0 \end{pmatrix}^T$

Numerical Example: Next-Character Prediction

The Task

Given the input character "e" and the memory of having just seen "h", predict the next character. Our simple vocabulary is `{'h':0, 'e':1, 'l':2, 'o':3}`.

Setup

We use one-hot encoding for characters. The layer has 2 hidden neurons.

- Input $x_{(t)}$ (char 'e'): $\begin{pmatrix} 0 & 1 & 0 & 0 \end{pmatrix}^T$
- Groundtruth Target $y_{(t)}$ (char 'l'): $\begin{pmatrix} 0 & 0 & 1 & 0 \end{pmatrix}^T$

Numerical Example: Next-Character Prediction

Setup

- Previous State $\mathrm{h}_{(t-1)}$ (from char 'h'): $\begin{pmatrix} 0.6 \\ 0.2 \end{pmatrix}$ *(Pre-computed for simplicity)*
- Weights & Biases: $\mathbf{W}_{xh} = \begin{pmatrix} 0.1 & 0.4 \\ 0.2 & 0.5 \\ 0.3 & 0.6 \\ 0.4 & 0.7 \end{pmatrix}$, $\mathbf{W}_{hh} = \begin{pmatrix} 0.1 & 0.5 \\ 0.5 & 0.1 \end{pmatrix}$, $\mathbf{W}_{hy} = \begin{pmatrix} 2 & 1 \\ -1 & 0 \\ 1 & 3 \\ 0 & -2 \end{pmatrix}$,

  $\mathrm{b}_h = \begin{pmatrix} 0.1 \\ 0.1 \end{pmatrix}$, $\mathrm{b}_y = \begin{pmatrix} 0.1 \\ 0.2 \\ 0.3 \\ 0.4 \end{pmatrix}$

Numerical Example: Next-Character Prediction

Step 1: Calculate the New Hidden State $\mathbf{h}_{(t)}$

$$\mathrm{h}_{(t)} = \tanh(\mathbf{W}_{xh}^{T}\mathrm{x}_{(t)} + \mathbf{W}_{hh}\mathrm{h}_{(t-1)} + \mathrm{b}_{h})$$

Numerical Example: Next-Character Prediction

Step 1: Calculate the New Hidden State $\mathbf{h}_{(t)}$

$$\mathrm{h}_{(t)} = \tanh(\mathbf{W}_{xh}^{T}\mathrm{x}_{(t)} + \mathbf{W}_{hh}\mathrm{h}_{(t-1)} + \mathrm{b}_h)$$

$$= \tanh\left(\underbrace{\begin{pmatrix} 0.1 & 0.2 & 0.3 & 0.4 \\ 0.4 & 0.5 & 0.6 & 0.7 \end{pmatrix}\begin{pmatrix} 0 \\ 1 \\ 0 \\ 0 \end{pmatrix}}_{\text{Input 'e'}} + \underbrace{\begin{pmatrix} 0.1 & 0.5 \\ 0.5 & 0.1 \end{pmatrix}\begin{pmatrix} 0.6 \\ 0.2 \end{pmatrix}}_{\text{Memory of 'h'}} + \begin{pmatrix} 0.1 \\ 0.1 \end{pmatrix}\right)$$

Numerical Example: Next-Character Prediction

Step 1: Calculate the New Hidden State $\mathbf{h}_{(t)}$

$$\mathrm{h}_{(t)} = \tanh(\mathbf{W}_{xh}^T \mathrm{x}_{(t)} + \mathbf{W}_{hh}\mathrm{h}_{(t-1)} + \mathrm{b}_h)$$

$$= \tanh\left(\underbrace{\begin{pmatrix} 0.1 & 0.2 & 0.3 & 0.4 \\ 0.4 & 0.5 & 0.6 & 0.7 \end{pmatrix} \begin{pmatrix} 0 \\ 1 \\ 0 \\ 0 \end{pmatrix}}_{\text{Input 'e'}} + \underbrace{\begin{pmatrix} 0.1 & 0.5 \\ 0.5 & 0.1 \end{pmatrix} \begin{pmatrix} 0.6 \\ 0.2 \end{pmatrix}}_{\text{Memory of 'h'}} + \begin{pmatrix} 0.1 \\ 0.1 \end{pmatrix} \right)$$

$$= \tanh\left(\begin{pmatrix} 0.2 \\ 0.5 \end{pmatrix} + \begin{pmatrix} 0.16 \\ 0.32 \end{pmatrix} + \begin{pmatrix} 0.1 \\ 0.1 \end{pmatrix} \right)$$

Numerical Example: Next-Character Prediction

Step 1: Calculate the New Hidden State $\mathbf{h}_{(t)}$

$$\mathrm{h}_{(t)} = \tanh(\mathbf{W}_{xh}^{T}\mathrm{x}_{(t)} + \mathbf{W}_{hh}\mathrm{h}_{(t-1)} + \mathrm{b}_h)$$

$$= \tanh\left(\underbrace{\begin{pmatrix} 0.1 & 0.2 & 0.3 & 0.4 \\ 0.4 & 0.5 & 0.6 & 0.7 \end{pmatrix}\begin{pmatrix} 0 \\ 1 \\ 0 \\ 0 \end{pmatrix}}_{\text{Input 'e'}} + \underbrace{\begin{pmatrix} 0.1 & 0.5 \\ 0.5 & 0.1 \end{pmatrix}\begin{pmatrix} 0.6 \\ 0.2 \end{pmatrix}}_{\text{Memory of 'h'}} + \begin{pmatrix} 0.1 \\ 0.1 \end{pmatrix}\right)$$

$$= \tanh\left(\begin{pmatrix} 0.2 \\ 0.5 \end{pmatrix} + \begin{pmatrix} 0.16 \\ 0.32 \end{pmatrix} + \begin{pmatrix} 0.1 \\ 0.1 \end{pmatrix}\right) = \tanh\begin{pmatrix} 0.46 \\ 0.92 \end{pmatrix}$$

Numerical Example: Next-Character Prediction

Step 1: Calculate the New Hidden State $\mathbf{h}_{(t)}$

$$\mathrm{h}_{(t)} = \tanh(\mathbf{W}_{xh}^T \mathrm{x}_{(t)} + \mathbf{W}_{hh}\mathrm{h}_{(t-1)} + \mathrm{b}_h)$$

$$= \tanh\left(\underbrace{\begin{pmatrix} 0.1 & 0.2 & 0.3 & 0.4 \\ 0.4 & 0.5 & 0.6 & 0.7 \end{pmatrix} \begin{pmatrix} 0 \\ 1 \\ 0 \\ 0 \end{pmatrix}}_{\text{Input 'e'}} + \underbrace{\begin{pmatrix} 0.1 & 0.5 \\ 0.5 & 0.1 \end{pmatrix} \begin{pmatrix} 0.6 \\ 0.2 \end{pmatrix}}_{\text{Memory of 'h'}} + \begin{pmatrix} 0.1 \\ 0.1 \end{pmatrix} \right)$$

$$= \tanh\left(\begin{pmatrix} 0.2 \\ 0.5 \end{pmatrix} + \begin{pmatrix} 0.16 \\ 0.32 \end{pmatrix} + \begin{pmatrix} 0.1 \\ 0.1 \end{pmatrix} \right) = \tanh \begin{pmatrix} 0.46 \\ 0.92 \end{pmatrix} = \begin{pmatrix} 0.43 \\ 0.72 \end{pmatrix}$$

Numerical Example: Next-Character Prediction

Step 2: Calculate Output Logits and Predict

The output $\mathrm{y}_{(t)}$ gives a score (logit) for each character in the vocabulary.

Numerical Example: Next-Character Prediction

Step 2: Calculate Output Logits and Predict

The output $\mathrm{y}_{(t)}$ gives a score (logit) for each character in the vocabulary.

$$\mathrm{y}_{(t)} = \mathbf{W}_{hy}^{T}\mathrm{h}_{(t)} + \mathrm{b}_y = \begin{pmatrix} 2 & -1 & 1 & 0 \\ 1 & 0 & 3 & -2 \end{pmatrix} \begin{pmatrix} 0.43 \\ 0.72 \end{pmatrix} + \begin{pmatrix} 0.1 \\ 0.2 \\ 0.3 \\ 0.4 \end{pmatrix}$$

Numerical Example: Next-Character Prediction

Step 2: Calculate Output Logits and Predict

The output $\mathrm{y}_{(t)}$ gives a score (logit) for each character in the vocabulary.

$$\mathrm{y}_{(t)} = \mathbf{W}_{hy}^T \mathrm{h}_{(t)} + \mathrm{b}_y = \begin{pmatrix} 2 & -1 & 1 & 0 \\ 1 & 0 & 3 & -2 \end{pmatrix} \begin{pmatrix} 0.43 \\ 0.72 \end{pmatrix} + \begin{pmatrix} 0.1 \\ 0.2 \\ 0.3 \\ 0.4 \end{pmatrix}$$

$$= \begin{pmatrix} 0.14 \\ 0.43 \\ 2.59 \\ -1.01 \end{pmatrix} + \begin{pmatrix} 0.1 \\ 0.2 \\ 0.3 \\ 0.4 \end{pmatrix}$$

Numerical Example: Next-Character Prediction

Step 2: Calculate Output Logits and Predict

The output $\mathrm{y}_{(t)}$ gives a score (logit) for each character in the vocabulary.

$$\mathrm{y}_{(t)} = \mathbf{W}_{hy}^T \mathrm{h}_{(t)} + \mathrm{b}_y = \begin{pmatrix} 2 & -1 & 1 & 0 \\ 1 & 0 & 3 & -2 \end{pmatrix} \begin{pmatrix} 0.43 \\ 0.72 \end{pmatrix} + \begin{pmatrix} 0.1 \\ 0.2 \\ 0.3 \\ 0.4 \end{pmatrix}$$

$$= \begin{pmatrix} 0.14 \\ 0.43 \\ 2.59 \\ -1.01 \end{pmatrix} + \begin{pmatrix} 0.1 \\ 0.2 \\ 0.3 \\ 0.4 \end{pmatrix} = \begin{pmatrix} 0.24 \\ 0.63 \\ \mathbf{2.89} \\ -0.61 \end{pmatrix} \begin{array}{l} \leftarrow \text{logit for 'h'} \\ \leftarrow \text{logit for 'e'} \\ \leftarrow \text{logit for 'l'} \\ \leftarrow \text{logit for 'o'} \end{array}$$

Numerical Example: Next-Character Prediction

Step 2: Calculate Output Logits and Predict

The output $\mathrm{y}_{(t)}$ gives a score (logit) for each character in the vocabulary.

$$\mathrm{y}_{(t)} = \mathbf{W}_{hy}^T \mathrm{h}_{(t)} + \mathrm{b}_y = \begin{pmatrix} 2 & -1 & 1 & 0 \\ 1 & 0 & 3 & -2 \end{pmatrix} \begin{pmatrix} 0.43 \\ 0.72 \end{pmatrix} + \begin{pmatrix} 0.1 \\ 0.2 \\ 0.3 \\ 0.4 \end{pmatrix}$$

$$= \begin{pmatrix} 0.14 \\ 0.43 \\ 2.59 \\ -1.01 \end{pmatrix} + \begin{pmatrix} 0.1 \\ 0.2 \\ 0.3 \\ 0.4 \end{pmatrix} = \begin{pmatrix} 0.24 \\ 0.63 \\ \mathbf{2.89} \\ -0.61 \end{pmatrix} \begin{matrix} \leftarrow \text{logit for 'h'} \\ \leftarrow \text{logit for 'e'} \\ \leftarrow \text{logit for 'l'} \\ \leftarrow \text{logit for 'o'} \end{matrix}$$

The highest logit is 2.89, which corresponds to the character 'l'. **The model predicts 'l'!**

Backpropagation in RNNs: The Challenge

The Problem: Dependencies Through Time

- The hidden state h_t at any time step t depends on the state from the previous step, h_{t-1}.

Backpropagation in RNNs: The Challenge

The Problem: Dependencies Through Time

- The hidden state h_t at any time step t depends on the state from the previous step, h_{t-1}.
- This creates a long computational graph where the loss at the end of a sequence depends on computations from every single time step.

Backpropagation in RNNs: The Challenge

The Problem: Dependencies Through Time

- The hidden state h_t at any time step t depends on the state from the previous step, h_{t-1}.
- This creates a long computational graph where the loss at the end of a sequence depends on computations from every single time step.

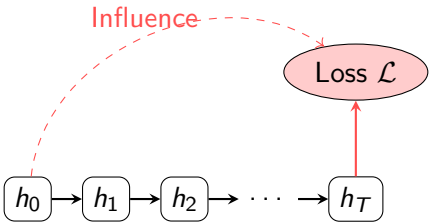
$$h_t \text{ depends on } h_{t-1}, \text{ which depends on } h_{t-2}, \text{ and so on...}$$

Backpropagation in RNNs: The Challenge

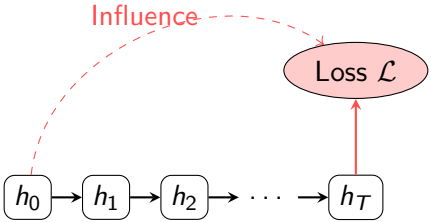
The Problem: Dependencies Through Time

- The hidden state h_t at any time step t depends on the state from the previous step, h_{t-1}.
- This creates a long computational graph where the loss at the end of a sequence depends on computations from every single time step.

$$h_t \text{ depends on } h_{t-1}, \text{ which depends on } h_{t-2}, \text{ and so on...}$$



The Main Challenge



The Core Issue

To calculate the gradient of the loss with respect to early parameters (like the weights used to compute h_1), we must backpropagate "through time", applying the chain rule across the entire sequence.

Backpropagation Through Time (BPTT)

What is BPTT?

BPTT is simply the application of the standard backpropagation algorithm to the "unrolled" computational graph of an RNN. It is not a new algorithm, but a name for the process.

Backpropagation Through Time (BPTT)

What is BPTT?

BPTT is simply the application of the standard backpropagation algorithm to the "unrolled" computational graph of an RNN. It is not a new algorithm, but a name for the process.

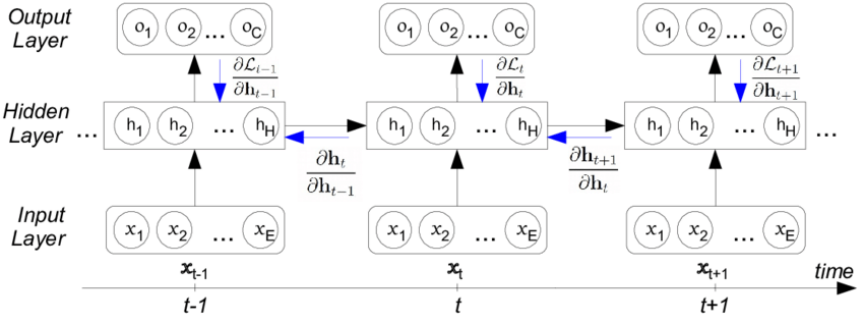


Figure: BPTT Illustration (source: DOI - 10.13140/RG.2.2.34411.08483)

Backpropagation Through Time (BPTT)

The Chain Rule in BPTT

The gradient of the final loss $\mathcal{L}$ with respect to the hidden state at time t, $\frac{\partial \mathcal{L}}{\partial h_t}$, has two components:

Backpropagation Through Time (BPTT)

The Chain Rule in BPTT

The gradient of the final loss $\mathcal{L}$ with respect to the hidden state at time t, $\frac{\partial \mathcal{L}}{\partial h_t}$, has two components:

1. The gradient from the output at the current time step (y_t).

Backpropagation Through Time (BPTT)

The Chain Rule in BPTT

The gradient of the final loss $\mathcal{L}$ with respect to the hidden state at time t, $\frac{\partial \mathcal{L}}{\partial h_t}$, has two components:

1. The gradient from the output at the current time step (y_t).
2. The gradient from the **next** hidden state (h_{t+1}), which was computed using h_t.

$$\frac{\partial \mathcal{L}}{\partial h_t} = \underbrace{\frac{\partial \mathcal{L}}{\partial y_t}\frac{\partial y_t}{\partial h_t}}_{\text{Current Loss}} + \underbrace{\frac{\partial \mathcal{L}}{\partial h_{t+1}}\frac{\partial h_{t+1}}{\partial h_t}}_{\text{Future Loss}}$$

Backpropagation Through Time (BPTT)

The Chain Rule in BPTT

The gradient of the final loss $\mathcal{L}$ with respect to the hidden state at time t, $\frac{\partial \mathcal{L}}{\partial h_t}$, has two components:

1. The gradient from the output at the current time step (y_t).
2. The gradient from the **next** hidden state (h_{t+1}), which was computed using h_t.

$$\frac{\partial \mathcal{L}}{\partial h_t} = \underbrace{\frac{\partial \mathcal{L}}{\partial y_t}\frac{\partial y_t}{\partial h_t}}_{\text{Current Loss}} + \underbrace{\frac{\partial \mathcal{L}}{\partial h_{t+1}}\frac{\partial h_{t+1}}{\partial h_t}}_{\text{Future Loss}}$$

This recursive formula is the heart of BPTT.

The Mathematics of Backpropagation Through Time

Objective

Our goal is to compute the gradients of the total loss $\mathcal{L} = \sum_{t=1}^{T} \mathcal{L}_t$ with respect to the model parameters: $\mathbf{W}_{xh}, \mathbf{W}_{hh}, \mathbf{W}_{hy}, \mathbf{b}_h, \mathbf{b}_y$. We use the chain rule to backpropagate the error from the end of the sequence to the beginning.

The Mathematics of Backpropagation Through Time

Objective

Our goal is to compute the gradients of the total loss $\mathcal{L} = \sum_{t=1}^{T} \mathcal{L}_t$ with respect to the model parameters: $\mathbf{W}_{xh}, \mathbf{W}_{hh}, \mathbf{W}_{hy}, \mathbf{b}_h, \mathbf{b}_y$. We use the chain rule to backpropagate the error from the end of the sequence to the beginning.

Step 1: The Chain Rule for Gradients

First, we compute the gradient of the loss with respect to the output and the hidden state at each time step t.

$$\frac{\partial \mathcal{L}}{\partial \mathbf{y}_t} = \frac{\partial \mathcal{L}_t}{\partial \mathbf{y}_t} \qquad \text{(From the loss function)}$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{h}_t} = \underbrace{\frac{\partial \mathcal{L}}{\partial \mathbf{y}_t}\frac{\partial \mathbf{y}_t}{\partial \mathbf{h}_t}}_{\text{From current output}} + \underbrace{\frac{\partial \mathcal{L}}{\partial \mathbf{h}_{t+1}}\frac{\partial \mathbf{h}_{t+1}}{\partial \mathbf{h}_t}}_{\text{From next hidden state}}$$

The Mathematics of Backpropagation Through Time

Step 2: Calculating Parameter Gradients

Once we have $\frac{\partial \mathcal{L}}{\partial \mathbf{h}_t}$, we can find the gradients for the weights and biases at that time step. The final gradient for a parameter is the sum of its gradients from all time steps.

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_{hy}} = \sum_{t=1}^{T} \frac{\partial \mathcal{L}}{\partial \mathbf{y}_t} \frac{\partial \mathbf{y}_t}{\partial \mathbf{W}_{hy}} = \sum_{t=1}^{T} \left(\frac{\partial \mathcal{L}_t}{\partial \mathbf{y}_t} \right)^T \mathbf{h}_t$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_{xh}} = \sum_{t=1}^{T} \frac{\partial \mathcal{L}}{\partial \mathbf{h}_t} \frac{\partial \mathbf{h}_t}{\partial \mathbf{W}_{xh}} = \sum_{t=1}^{T} \text{diag}(\phi'(\dots))^T \left(\frac{\partial \mathcal{L}}{\partial \mathbf{h}_t} \right) \mathbf{x}_t^T$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_{hh}} = \sum_{t=1}^{T} \frac{\partial \mathcal{L}}{\partial \mathbf{h}_t} \frac{\partial \mathbf{h}_t}{\partial \mathbf{W}_{hh}} = \sum_{t=1}^{T} \text{diag}(\phi'(\dots))^T \left(\frac{\partial \mathcal{L}}{\partial \mathbf{h}_t} \right) \mathbf{h}_{t-1}^T$$

BPTT: Numerical Example

Setup

Consider a 1-neuron RNN with a sequence of length 2. Let loss $\mathcal{L}_t = (y_t - \text{target}_t)^2$.

- Inputs: $x_1 = 2, x_2 = 3$
- Initial State: $h_0 = 0$
- Weights: $W_{xh} = 0.5, W_{hh} = 0.2, W_{hy} = 0.4$ (Biases are 0 for simplicity)
- Activation: $\phi = \tanh$, so $\phi'(z) = 1 - \tanh^2(z)$
- Targets: $\text{target}_1 = 0.5, \text{target}_2 = 0.8$

BPTT: Numerical Example

Setup

Consider a 1-neuron RNN with a sequence of length 2. Let loss $\mathcal{L}_t = (y_t - \text{target}_t)^2$.

- Inputs: $x_1 = 2, x_2 = 3$
- Initial State: $h_0 = 0$
- Weights: $W_{xh} = 0.5, W_{hh} = 0.2, W_{hy} = 0.4$ (Biases are 0 for simplicity)
- Activation: $\phi = \tanh$, so $\phi'(z) = 1 - \tanh^2(z)$
- Targets: $\text{target}_1 = 0.5, \text{target}_2 = 0.8$

Step 1: Forward Pass

- **t=1**: $h_1 = \tanh(0.5 \cdot 2 + 0.2 \cdot 0) = \tanh(1) = 0.76$. $y_1 = 0.4 \cdot 0.76 = 0.304$.
  $\mathcal{L}_1 = (0.304 - 0.5)^2 = 0.038$.
- **t=2**: $h_2 = \tanh(0.5 \cdot 3 + 0.2 \cdot 0.76) = \tanh(1.652) = 0.93$. $y_2 = 0.4 \cdot 0.93 = 0.744$.
  $\mathcal{L}_2 = (0.744 - 0.8)^2 = 0.003$.

BPTT: Numerical Example

Step 2: Backward Pass (starting from the end)

Total Loss $\mathcal{L} = 0.038 + 0.003 = 0.041$.

- **t=2**: $\frac{\partial \mathcal{L}}{\partial y_2} = 2(y_2 - \text{target}_2) = 2(0.744 - 0.8) = -0.112$.
  $\frac{\partial \mathcal{L}}{\partial h_2} = \frac{\partial \mathcal{L}}{\partial y_2}\frac{\partial y_2}{\partial h_2} = -0.112 \cdot W_{hy} = -0.112 \cdot 0.4 = -0.045$.
  $\frac{\partial \mathcal{L}_2}{\partial W_{hh}} = \frac{\partial \mathcal{L}}{\partial h_2}(1 - h_2^2)h_1 = -0.045 \cdot (1 - 0.93^2) \cdot 0.76 = -0.0046$.

BPTT: Numerical Example

Step 2: Backward Pass (starting from the end)

Total Loss $\mathcal{L} = 0.038 + 0.003 = 0.041$.

- **t=2**: $\frac{\partial \mathcal{L}}{\partial y_2} = 2(y_2 - \text{target}_2) = 2(0.744 - 0.8) = -0.112$.
  $\frac{\partial \mathcal{L}}{\partial h_2} = \frac{\partial \mathcal{L}}{\partial y_2}\frac{\partial y_2}{\partial h_2} = -0.112 \cdot W_{hy} = -0.112 \cdot 0.4 = -0.045$.
  $\frac{\partial \mathcal{L}_2}{\partial W_{hh}} = \frac{\partial \mathcal{L}}{\partial h_2}(1 - h_2^2)h_1 = -0.045 \cdot (1 - 0.93^2) \cdot 0.76 = -0.0046$.

- **t=1**: $\frac{\partial \mathcal{L}}{\partial y_1} = 2(y_1 - \text{target}_1) = 2(0.304 - 0.5) = -0.392$.
  The crucial step:
  $\frac{\partial \mathcal{L}}{\partial h_1} = (\frac{\partial \mathcal{L}}{\partial y_1} W_{hy}) + (\frac{\partial \mathcal{L}}{\partial h_2}\frac{\partial h_2}{\partial h_1})$. $\frac{\partial h_2}{\partial h_1} = (1 - h_2^2)W_{hh} = (1 - 0.93^2) \cdot 0.2 = 0.027$.
  $\frac{\partial \mathcal{L}}{\partial h_1} = (-0.392 \cdot 0.4) + (-0.045 \cdot 0.027) = -0.1568 - 0.0012 = -0.158$.
  $\frac{\partial \mathcal{L}_1}{\partial W_{hh}} = \frac{\partial \mathcal{L}}{\partial h_1}(1 - h_1^2)h_0 = -0.158 \cdot (1 - 0.76^2) \cdot 0 = 0$.

Final Gradient: $\frac{\partial \mathcal{L}}{\partial W_{hh}} = \frac{\partial \mathcal{L}_1}{\partial W_{hh}} + \frac{\partial \mathcal{L}_2}{\partial W_{hh}} = 0 - 0.0046 = -0.0046$.

Numerical Instability in BPTT

The Problem with Long Sequences

For a sequence of length T, calculating the gradient for the initial weights involves a chain rule product of T matrices.

$$\frac{\partial h_T}{\partial h_0} = \prod_{t=1}^{T} \frac{\partial h_t}{\partial h_{t-1}} = \prod_{t=1}^{T} \mathbf{W}_{hh}^T \text{diag}(\phi'(\ldots))$$

This long product is the source of major numerical instability.

The Twin Problems: Exploding and Vanishing Gradients

Gradient Explosion

- **What happens?** If the recurrent weight matrix $\mathbf{W}_{hh}$ has large values, the long product of matrices can grow exponentially.

The Twin Problems: Exploding and Vanishing Gradients

Gradient Explosion

- **What happens?** If the recurrent weight matrix $\mathbf{W}_{hh}$ has large values, the long product of matrices can grow exponentially.
- **Result:** Gradients become enormous (NaN or infinity).

The Twin Problems: Exploding and Vanishing Gradients

Gradient Explosion

- **What happens?** If the recurrent weight matrix $\mathbf{W}_{hh}$ has large values, the long product of matrices can grow exponentially.
- **Result:** Gradients become enormous (NaN or infinity).
- **Effect:** Model weights are updated by huge amounts.

The Twin Problems: Exploding and Vanishing Gradients

Gradient Explosion

- **What happens?** If the recurrent weight matrix $\mathbf{W}_{hh}$ has large values, the long product of matrices can grow exponentially.
- **Result:** Gradients become enormous (NaN or infinity).
- **Effect:** Model weights are updated by huge amounts.
- **Solution: Gradient Clipping**. If the norm of the gradient exceeds a certain threshold, it is scaled down.

The Twin Problems: Exploding and Vanishing Gradients

Gradient Explosion

- **What happens?** If the recurrent weight matrix $\mathbf{W}_{hh}$ has large values, the long product of matrices can grow exponentially.
- **Result:** Gradients become enormous (NaN or infinity).
- **Effect:** Model weights are updated by huge amounts.
- **Solution: Gradient Clipping**. If the norm of the gradient exceeds a certain threshold, it is scaled down.

Gradient Vanishing

- **What happens?** If the recurrent weight matrix has small values, the long product of matrices can shrink exponentially towards zero.

The Twin Problems: Exploding and Vanishing Gradients

Gradient Explosion

- **What happens?** If the recurrent weight matrix $\mathbf{W}_{hh}$ has large values, the long product of matrices can grow exponentially.
- **Result:** Gradients become enormous (NaN or infinity).
- **Effect:** Model weights are updated by huge amounts.
- **Solution: Gradient Clipping**. If the norm of the gradient exceeds a certain threshold, it is scaled down.

Gradient Vanishing

- **What happens?** If the recurrent weight matrix has small values, the long product of matrices can shrink exponentially towards zero.
- **Result:** Gradients for early time steps become nearly zero.

The Twin Problems: Exploding and Vanishing Gradients

Gradient Explosion

- **What happens?** If the recurrent weight matrix $\mathbf{W}_{hh}$ has large values, the long product of matrices can grow exponentially.
- **Result:** Gradients become enormous (NaN or infinity).
- **Effect:** Model weights are updated by huge amounts.
- **Solution: Gradient Clipping**. If the norm of the gradient exceeds a certain threshold, it is scaled down.

Gradient Vanishing

- **What happens?** If the recurrent weight matrix has small values, the long product of matrices can shrink exponentially towards zero.
- **Result:** Gradients for early time steps become nearly zero.
- **Effect:** The model cannot learn long-range dependencies.

The Twin Problems: Exploding and Vanishing Gradients

Gradient Explosion

- **What happens?** If the recurrent weight matrix $\mathbf{W}_{hh}$ has large values, the long product of matrices can grow exponentially.
- **Result:** Gradients become enormous (NaN or infinity).
- **Effect:** Model weights are updated by huge amounts.
- **Solution: Gradient Clipping**. If the norm of the gradient exceeds a certain threshold, it is scaled down.

Gradient Vanishing

- **What happens?** If the recurrent weight matrix has small values, the long product of matrices can shrink exponentially towards zero.
- **Result:** Gradients for early time steps become nearly zero.
- **Effect:** The model cannot learn long-range dependencies.
- **Solution:** More complex RNN architectures.

Truncated BPTT: A Practical Compromise

The Problem

Running BPTT on very long sequences is computationally expensive and memory-intensive, and it exacerbates the gradient explosion/vanishing problems.

Truncated BPTT: A Practical Compromise

The Problem

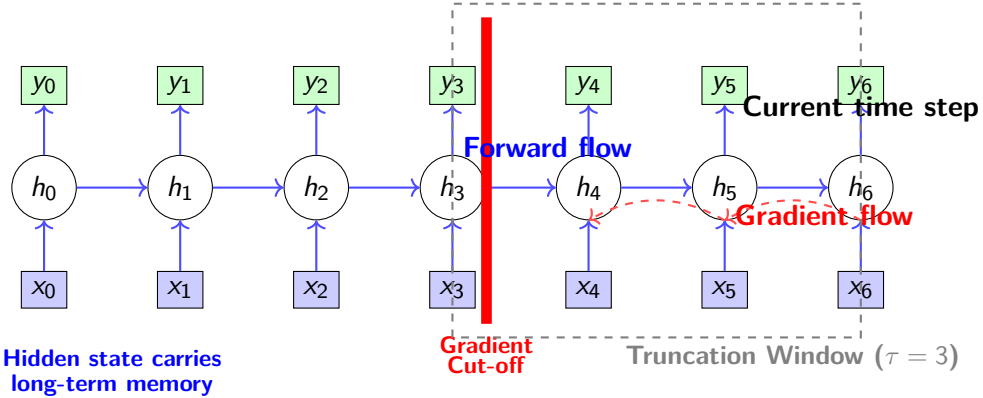
Running BPTT on very long sequences is computationally expensive and memory-intensive, and it exacerbates the gradient explosion/vanishing problems.

The Solution: Truncated BPTT

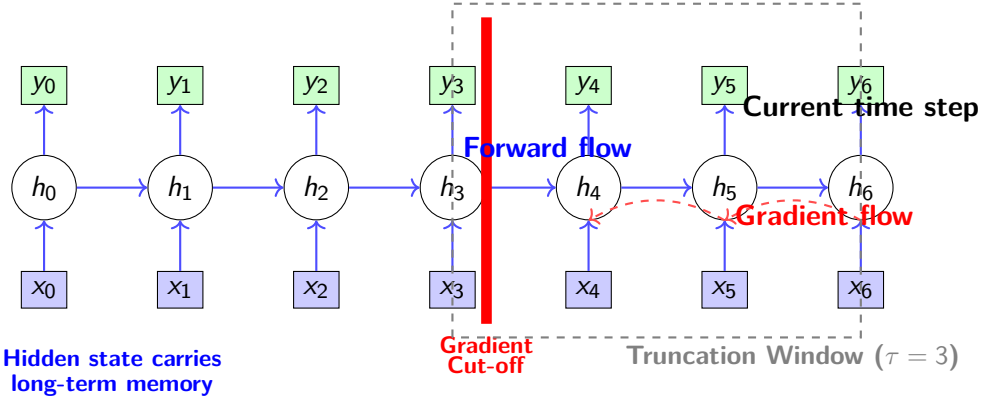
Instead of backpropagating through the entire sequence, we detach the gradient history after a fixed number of steps (τ).

- The model still propagates its hidden state forward through the whole sequence, maintaining its long-term memory.
- However, during the backward pass, the gradient calculation is cut off after τ steps.

Truncated BPTT: Illustration



Truncated BPTT: Illustration



Trade-off

It limits the model's ability to learn dependencies that span longer than τ time steps.

Advanced Memory based Models

Introduction to LSTM and GRU

Training RNNs: Challenges

1. The Vanishing Gradient Problem

- During backpropagation, we repeatedly multiply gradients by the same recurrent weight matrix.

Training RNNs: Challenges

1. The Vanishing Gradient Problem

- During backpropagation, we repeatedly multiply gradients by the same recurrent weight matrix.
- Hence for **small weights**, gradient signal **shrinks** eponentially.

Training RNNs: Challenges

1. The Vanishing Gradient Problem

- During backpropagation, we repeatedly multiply gradients by the same recurrent weight matrix.
- Hence for **small weights**, gradient signal **shrinks** eponentially.
- As a result the network does not learn long -term dependencies.

Training RNNs: Challenges

1. The Vanishing Gradient Problem

- During backpropagation, we repeatedly multiply gradients by the same recurrent weight matrix.
- Hence for **small weights**, gradient signal **shrinks** eponentially.
- As a result the network does not learn long -term dependencies.
- The model forgets what happened early in the sequence.

Training RNNs: Challenges

1. The Vanishing Gradient Problem

- During backpropagation, we repeatedly multiply gradients by the same recurrent weight matrix.
- Hence for **small weights**, gradient signal **shrinks** eponentially.
- As a result the network does not learn long -term dependencies.
- The model forgets what happened early in the sequence.

2. The Exploding Gradient Problem

- During backpropagation, we repeatedly multiply gradients by the same recurrent weight matrix.

Training RNNs: Challenges

1. The Vanishing Gradient Problem

- During backpropagation, we repeatedly multiply gradients by the same recurrent weight matrix.
- Hence for **small weights**, gradient signal **shrinks** eponentially.
- As a result the network does not learn long -term dependencies.
- The model forgets what happened early in the sequence.

2. The Exploding Gradient Problem

- During backpropagation, we repeatedly multiply gradients by the same recurrent weight matrix.
- Hence for **large weights**, gradient signal **grows** exponentially.

Training RNNs: Challenges

1. The Vanishing Gradient Problem

- During backpropagation, we repeatedly multiply gradients by the same recurrent weight matrix.
- Hence for **small weights**, gradient signal **shrinks** eponentially.
- As a result the network does not learn long -term dependencies.
- The model forgets what happened early in the sequence.

2. The Exploding Gradient Problem

- During backpropagation, we repeatedly multiply gradients by the same recurrent weight matrix.
- Hence for **large weights**, gradient signal **grows** exponentially.
- It leads to massive weight updates.

Training RNNs: Challenges

1. The Vanishing Gradient Problem

- During backpropagation, we repeatedly multiply gradients by the same recurrent weight matrix.
- Hence for **small weights**, gradient signal **shrinks** eponentially.
- As a result the network does not learn long -term dependencies.
- The model forgets what happened early in the sequence.

2. The Exploding Gradient Problem

- During backpropagation, we repeatedly multiply gradients by the same recurrent weight matrix.
- Hence for **large weights**, gradient signal **grows** exponentially.
- It leads to massive weight updates.
- Hence it results in Training unstability (often resulting in NaN values).

The Solution: Long Short-Term Memory (LSTM)

What is an LSTM?

LSTM networks are a type of recurrent neural network (RNN) designed to solve the vanishing gradient problem. They are particularly effective at learning long-term dependencies in sequential data.

The Solution: Long Short-Term Memory (LSTM)

What is an LSTM?

LSTM networks are a type of recurrent neural network (RNN) designed to solve the vanishing gradient problem. They are particularly effective at learning long-term dependencies in sequential data.

Key Differences from Simple RNNs

- **Dual States**: LSTMs have two hidden states: a short-term state $\mathbf{h_t}$ and a long-term state (or cell state) $\mathbf{c_t}$, enhancing processing of longer sequences.

The Solution: Long Short-Term Memory (LSTM)

What is an LSTM?

LSTM networks are a type of recurrent neural network (RNN) designed to solve the vanishing gradient problem. They are particularly effective at learning long-term dependencies in sequential data.

Key Differences from Simple RNNs

- **Dual States**: LSTMs have two hidden states: a short-term state $\mathbf{h_t}$ and a long-term state (or cell state) $\mathbf{c_t}$, enhancing processing of longer sequences.
- **Gated Mechanism**: They use special "gates" to regulate the flow of information into and out of the cell state.

The Solution: Long Short-Term Memory (LSTM)

What is an LSTM?

LSTM networks are a type of recurrent neural network (RNN) designed to solve the vanishing gradient problem. They are particularly effective at learning long-term dependencies in sequential data.

Key Differences from Simple RNNs

- **Dual States**: LSTMs have two hidden states: a short-term state $\mathbf{h_t}$ and a long-term state (or cell state) $\mathbf{c_t}$, enhancing processing of longer sequences.
- **Gated Mechanism**: They use special "gates" to regulate the flow of information into and out of the cell state.
- **Gradient Stability**: The gates and additive interactions within the cell state help combat both vanishing and exploding gradients, making training more stable.

LSTM Architecture: The Gates

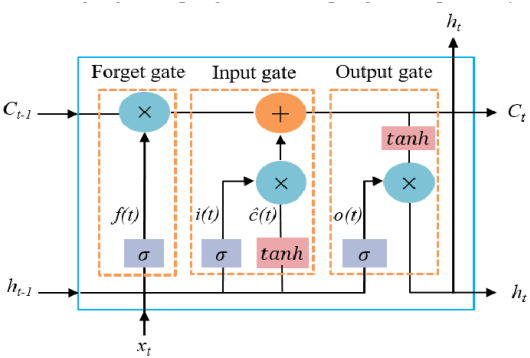


Figure: LSTM architecture (source: DOI-10.1109/ACCESS.2021.3125733)

The Core Components

- **Forget Gate:** Decides what information from the old cell state to throw away.

LSTM Architecture: The Gates

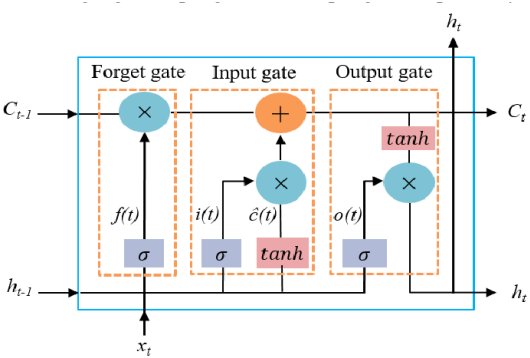


Figure: LSTM architecture (source: DOI-10.1109/ACCESS.2021.3125733)

The Core Components

- **Forget Gate:** Decides what information from the old cell state to throw away.
- **Input Gate:** Decides what new information from the current input to store in the cell state.

LSTM Architecture: The Gates

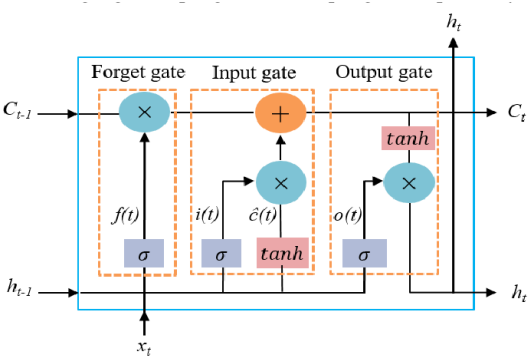


Figure: LSTM architecture (source: DOI-10.1109/ACCESS.2021.3125733)

The Core Components

- **Forget Gate:** Decides what information from the old cell state to throw away.
- **Input Gate:** Decides what new information from the current input to store in the cell state.
- **Output Gate:** Decides what part of the cell state to output as the new hidden state.

LSTM Equations: The Forget Gate

Function and Intuition

The forget gate determines how much of the old cell state, $\mathbf{c_{t-1}}$, should be forgotten. A sigmoid function σ outputs values between 0 and 1, where 1 means "keep this information completely" and 0 means "forget it completely."

LSTM Equations: The Forget Gate

Function and Intuition

The forget gate determines how much of the old cell state, $\mathbf{c_{t-1}}$, should be forgotten. A sigmoid function σ outputs values between 0 and 1, where 1 means "keep this information completely" and 0 means "forget it completely."

- **Mathematical Equation:**

$$\mathbf{f_t} = \sigma(\mathbf{W}_f \cdot [\mathbf{h_{t-1}}, \mathbf{x_t}] + \mathbf{b}_f)$$

LSTM Equations: The Forget Gate

Function and Intuition

The forget gate determines how much of the old cell state, $\mathbf{c_{t-1}}$, should be forgotten. A sigmoid function σ outputs values between 0 and 1, where 1 means "keep this information completely" and 0 means "forget it completely."

- **Mathematical Equation:**

$$\mathbf{f_t} = \sigma(\mathbf{W}_f \cdot [\mathbf{h_{t-1}}, \mathbf{x_t}] + \mathbf{b}_f)$$

- **Explanation:**
  - $\mathbf{W}_f$: Weight matrix for the forget gate.
  - $\mathbf{x_t}$: The current input.
  - $\mathbf{h_{t-1}}$: The previous hidden state.
  - $[\mathbf{h_{t-1}}, \mathbf{x_t}]$: Concatenation of the previous hidden state and the current input.
  - $\mathbf{b}_f$: Bias vector for the forget gate.

LSTM Equations: The Forget Gate

Function and Intuition

The forget gate determines how much of the old cell state, $\mathbf{c_{t-1}}$, should be forgotten. A sigmoid function σ outputs values between 0 and 1, where 1 means "keep this information completely" and 0 means "forget it completely."

- **Mathematical Equation:**

$$\mathbf{f_t} = \sigma(\mathbf{W}_f \cdot [\mathbf{h_{t-1}}, \mathbf{x_t}] + \mathbf{b}_f)$$

- **Explanation:**
  - $\mathbf{W}_f$: Weight matrix for the forget gate.
  - $\mathbf{x_t}$: The current input.
  - $\mathbf{h_{t-1}}$: The previous hidden state.
  - $[\mathbf{h_{t-1}}, \mathbf{x_t}]$: Concatenation of the previous hidden state and the current input.
  - $\mathbf{b}_f$: Bias vector for the forget gate.
- **Interpretation:** The forget gate learns to selectively discard irrelevant past information based on the current input and previous hidden state.

LSTM Equations: The Input Gate

Function and Intuition

The input gate decides which new information from the current input is relevant and should be stored in the cell state. This is a two-step process involving an input gate and a candidate gate.

LSTM Equations: The Input Gate

Function and Intuition

The input gate decides which new information from the current input is relevant and should be stored in the cell state. This is a two-step process involving an input gate and a candidate gate.

- **Input Gate Equation:**

$$\mathbf{i_t} = \sigma(\mathbf{W}_i \cdot [\mathbf{h_{t-1}}, \mathbf{x_t}] + \mathbf{b}_i)$$

- **Candidate Gate Equation:**

$$\tilde{\mathbf{c}}_\mathbf{t} = \tanh(\mathbf{W}_c \cdot [\mathbf{h_{t-1}}, \mathbf{x_t}] + \mathbf{b}_c)$$

LSTM Equations: The Input Gate

Function and Intuition

The input gate decides which new information from the current input is relevant and should be stored in the cell state. This is a two-step process involving an input gate and a candidate gate.

- **Input Gate Equation:**

$$\mathbf{i_t} = \sigma(\mathbf{W}_i \cdot [\mathbf{h_{t-1}}, \mathbf{x_t}] + \mathbf{b}_i)$$

- **Candidate Gate Equation:**

$$\tilde{\mathbf{c}}_\mathbf{t} = \tanh(\mathbf{W}_c \cdot [\mathbf{h_{t-1}}, \mathbf{x_t}] + \mathbf{b}_c)$$

- **Interpretation:**
  - The input gate ($\mathbf{i_t}$) acts as a filter, deciding which values from the candidate vector are important.

LSTM Equations: The Input Gate

Function and Intuition

The input gate decides which new information from the current input is relevant and should be stored in the cell state. This is a two-step process involving an input gate and a candidate gate.

- **Input Gate Equation:**

$$\mathbf{i_t} = \sigma(\mathbf{W}_i \cdot [\mathbf{h_{t-1}}, \mathbf{x_t}] + \mathbf{b}_i)$$

- **Candidate Gate Equation:**

$$\tilde{\mathbf{c}}_\mathbf{t} = \tanh(\mathbf{W}_c \cdot [\mathbf{h_{t-1}}, \mathbf{x_t}] + \mathbf{b}_c)$$

- **Interpretation:**
  - The input gate ($\mathbf{i_t}$) acts as a filter, deciding which values from the candidate vector are important.
  - The candidate gate ($\tilde{\mathbf{c}}_\mathbf{t}$) uses a tanh function to create a new vector of potential information, scaled between -1 and 1.

LSTM Equations: The Input Gate

Function and Intuition

The input gate decides which new information from the current input is relevant and should be stored in the cell state. This is a two-step process involving an input gate and a candidate gate.

- **Input Gate Equation:**

$$\mathbf{i_t} = \sigma(\mathbf{W}_i \cdot [\mathbf{h_{t-1}}, \mathbf{x_t}] + \mathbf{b}_i)$$

- **Candidate Gate Equation:**

$$\tilde{\mathbf{c}}_\mathbf{t} = \tanh(\mathbf{W}_c \cdot [\mathbf{h_{t-1}}, \mathbf{x_t}] + \mathbf{b}_c)$$

- **Interpretation:**
  - The input gate ($\mathbf{i_t}$) acts as a filter, deciding which values from the candidate vector are important.
  - The candidate gate ($\tilde{\mathbf{c}}_\mathbf{t}$) uses a tanh function to create a new vector of potential information, scaled between -1 and 1.
  - These two are multiplied element-wise to form the new information that will be added to the cell state.

LSTM Equations: Updating the Cell State

Function and Intuition

This is the core of the LSTM's memory mechanism. The new cell state $\mathbf{c_t}$ is a combination of the old cell state, which has been partially forgotten, and the new information, which has been carefully selected.

LSTM Equations: Updating the Cell State

Function and Intuition

This is the core of the LSTM's memory mechanism. The new cell state $\mathbf{c_t}$ is a combination of the old cell state, which has been partially forgotten, and the new information, which has been carefully selected.

- **Equation:**

$$\mathbf{c_t} = \mathbf{f_t} \odot \mathbf{c_{t-1}} + \mathbf{i_t} \odot \mathbf{\epsilon_t}$$

LSTM Equations: Updating the Cell State

Function and Intuition

This is the core of the LSTM's memory mechanism. The new cell state $\mathbf{c_t}$ is a combination of the old cell state, which has been partially forgotten, and the new information, which has been carefully selected.

- **Equation:**

$$\mathbf{c_t} = \mathbf{f_t} \odot \mathbf{c_{t-1}} + \mathbf{i_t} \odot \boldsymbol{\epsilon}_\mathbf{t}$$

- **Explanation:**
  - $\odot$: Element-wise multiplication.
  - The first term, $\mathbf{f_t} \odot \mathbf{c_{t-1}}$, represents the old memory with some parts forgotten.
  - The second term, $\mathbf{i_t} \odot \boldsymbol{\epsilon}_\mathbf{t}$, represents the new information being added.

LSTM Equations: Updating the Cell State

Function and Intuition

This is the core of the LSTM's memory mechanism. The new cell state $\mathbf{c_t}$ is a combination of the old cell state, which has been partially forgotten, and the new information, which has been carefully selected.

- **Equation:**

$$\mathbf{c_t} = \mathbf{f_t} \odot \mathbf{c_{t-1}} + \mathbf{i_t} \odot \boldsymbol{\epsilon}_\mathbf{t}$$

- **Explanation:**
  - $\odot$: Element-wise multiplication.
  - The first term, $\mathbf{f_t} \odot \mathbf{c_{t-1}}$, represents the old memory with some parts forgotten.
  - The second term, $\mathbf{i_t} \odot \boldsymbol{\epsilon}_\mathbf{t}$, represents the new information being added.
- **Interpretation:** This additive interaction is crucial for preserving gradients. Instead of a series of multiplicative steps, the addition allows gradients to flow directly back, preventing them from vanishing.

LSTM Equations: The Output Gate

Function and Intuition

The output gate decides what part of the cell state will be used to compute the hidden state $\mathbf{h_t}$. This allows the network to expose only the relevant information from its long-term memory.

LSTM Equations: The Output Gate

Function and Intuition

The output gate decides what part of the cell state will be used to compute the hidden state $\mathbf{h_t}$. This allows the network to expose only the relevant information from its long-term memory.

- **Output Gate Equation:**

$$\mathbf{o_t} = \sigma(\mathbf{W}_o \cdot [\mathbf{h_{t-1}}, \mathbf{x_t}] + \mathbf{b}_o)$$

LSTM Equations: The Output Gate

Function and Intuition

The output gate decides what part of the cell state will be used to compute the hidden state $\mathbf{h_t}$. This allows the network to expose only the relevant information from its long-term memory.

- **Output Gate Equation:**

$$\mathbf{o_t} = \sigma(\mathbf{W}_o \cdot [\mathbf{h_{t-1}}, \mathbf{x_t}] + \mathbf{b}_o)$$

- **New Hidden State Equation:**

$$\mathbf{h_t} = \mathbf{o_t} \odot \tanh(\mathbf{c_t})$$

LSTM Equations: The Output Gate

Function and Intuition

The output gate decides what part of the cell state will be used to compute the hidden state $\mathbf{h_t}$. This allows the network to expose only the relevant information from its long-term memory.

- **Output Gate Equation:**

$$\mathbf{o_t} = \sigma(\mathbf{W}_o \cdot [\mathbf{h_{t-1}}, \mathbf{x_t}] + \mathbf{b}_o)$$

- **New Hidden State Equation:**

$$\mathbf{h_t} = \mathbf{o_t} \odot \tanh(\mathbf{c_t})$$

- **Interpretation:**
  - The hidden state $\mathbf{h_t}$ is a filtered version of the cell state.

LSTM Equations: The Output Gate

Function and Intuition

The output gate decides what part of the cell state will be used to compute the hidden state $\mathbf{h_t}$. This allows the network to expose only the relevant information from its long-term memory.

- **Output Gate Equation:**

$$\mathbf{o_t} = \sigma(\mathbf{W}_o \cdot [\mathbf{h_{t-1}}, \mathbf{x_t}] + \mathbf{b}_o)$$

- **New Hidden State Equation:**

$$\mathbf{h_t} = \mathbf{o_t} \odot \tanh(\mathbf{c_t})$$

- **Interpretation:**
  - The hidden state $\mathbf{h_t}$ is a filtered version of the cell state.
  - The tanh function squashes the cell state values to be between -1 and 1.

LSTM Equations: The Output Gate

Function and Intuition

The output gate decides what part of the cell state will be used to compute the hidden state $\mathbf{h_t}$. This allows the network to expose only the relevant information from its long-term memory.

- **Output Gate Equation:**

$$\mathbf{o_t} = \sigma(\mathbf{W}_o \cdot [\mathbf{h_{t-1}}, \mathbf{x_t}] + \mathbf{b}_o)$$

- **New Hidden State Equation:**

$$\mathbf{h_t} = \mathbf{o_t} \odot \tanh(\mathbf{c_t})$$

- **Interpretation:**
  - The hidden state $\mathbf{h_t}$ is a filtered version of the cell state.
  - The tanh function squashes the cell state values to be between -1 and 1.
  - This tanh output is then multiplied element-wise by the output gate $\mathbf{o_t}$, which selectively exposes the useful information from the cell state.

Introduction to GRU (Gated Recurrent Unit)

A Simpler Alternative to LSTM

The Gated Recurrent Unit (GRU), introduced in 2014, is a popular alternative to the LSTM. It aims to solve the same vanishing gradient problem but with a simpler architecture and fewer parameters.

Introduction to GRU (Gated Recurrent Unit)

A Simpler Alternative to LSTM

The Gated Recurrent Unit (GRU), introduced in 2014, is a popular alternative to the LSTM. It aims to solve the same vanishing gradient problem but with a simpler architecture and fewer parameters.

Key Differences from LSTM

- **Two Gates instead of Three:** GRU uses an **Update Gate (z_t)** and a **Reset Gate (r_t)**.
- **No Separate Cell State:** GRU merges the cell state and hidden state into a single hidden state vector, h_t.

Introduction to GRU (Gated Recurrent Unit)

A Simpler Alternative to LSTM

The Gated Recurrent Unit (GRU), introduced in 2014, is a popular alternative to the LSTM. It aims to solve the same vanishing gradient problem but with a simpler architecture and fewer parameters.

Key Differences from LSTM

- **Two Gates instead of Three:** GRU uses an **Update Gate (z_t)** and a **Reset Gate (r_t)**.
- **No Separate Cell State:** GRU merges the cell state and hidden state into a single hidden state vector, h_t.

Benefits

Due to its simpler design, a GRU often trains faster than an LSTM and can perform just as well, or sometimes even better, on less complex datasets.

GRU architecture: Gates

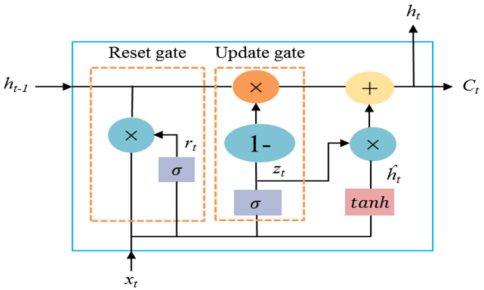


Figure: GRU Architecture (source: DOI-10.1007/s11042-023-15571-y)

The Core Components: Gating Mechanisms

- **Reset Gate (R_t):** Controls how much of the past information to forget.
- **Update Gate (Z_t):** Controls how much of the past information to carry forward.

GRU Architecture: The Reset Gate

Function: Controlling Short-Term Memory

The **Reset Gate ($\mathbf{R}_t$)** determines how the previous hidden state $\mathbf{H}_{t-1}$ is used to compute the *new candidate hidden state*.

- When entries in $\mathbf{R}_t$ are close to 1, the gate allows the previous hidden state to pass through, preserving its memory.
- When entries are close to 0, the gate effectively "resets" or ignores parts of the previous hidden state. This is useful when the current input is the start of a new, unrelated sequence of information.

GRU Architecture: The Reset Gate

Function: Controlling Short-Term Memory

The **Reset Gate ($\mathbf{R}_t$)** determines how the previous hidden state $\mathbf{H}_{t-1}$ is used to compute the *new candidate hidden state*.

- When entries in $\mathbf{R}_t$ are close to 1, the gate allows the previous hidden state to pass through, preserving its memory.
- When entries are close to 0, the gate effectively "resets" or ignores parts of the previous hidden state. This is useful when the current input is the start of a new, unrelated sequence of information.

Formula

$$\mathbf{R}_t = \sigma(\mathbf{X}_t\mathbf{W}_{xr} + \mathbf{H}_{t-1}\mathbf{W}_{hr} + \mathbf{b}_r)$$

GRU Architecture: The Update Gate

Function: Controlling Long-Term Memory

The **Update Gate ($\mathbf{Z}_t$)** controls how much of the previous hidden state $\mathbf{H}_{t-1}$ is directly carried over to the final hidden state $\mathbf{H}_t$.

- It acts similarly to the forget gate in an LSTM.
- When entries in $\mathbf{Z}_t$ are close to 1, the corresponding dimension in $\mathbf{H}_{t-1}$ is almost entirely copied to $\mathbf{H}_t$, preserving long-term memory.
- When entries are close to 0, the old information is discarded in favor of the new candidate state.

GRU Architecture: The Update Gate

Function: Controlling Long-Term Memory

The **Update Gate ($\mathbf{Z}_t$)** controls how much of the previous hidden state $\mathbf{H}_{t-1}$ is directly carried over to the final hidden state $\mathbf{H}_t$.

- It acts similarly to the forget gate in an LSTM.
- When entries in $\mathbf{Z}_t$ are close to 1, the corresponding dimension in $\mathbf{H}_{t-1}$ is almost entirely copied to $\mathbf{H}_t$, preserving long-term memory.
- When entries are close to 0, the old information is discarded in favor of the new candidate state.

Formula

$$\mathbf{Z}_t = \sigma(\mathbf{X}_t\mathbf{W}_{xz} + \mathbf{H}_{t-1}\mathbf{W}_{hz} + \mathbf{b}_z)$$

GRU Architecture: Hidden State

Step 1: The Candidate Hidden State ($\tilde{\mathbf{H}}_t$)

First, a candidate hidden state is created. It's similar to the hidden state in a simple RNN, but with a key difference: the reset gate $\mathbf{R}_t$ controls how much of the previous state $\mathbf{H}_{t-1}$ influences this new candidate.

$$\tilde{\mathbf{H}}_t = \tanh(\mathbf{X}_t \mathbf{W}_{xh} + (\mathbf{R}_t \odot \mathbf{H}_{t-1})\mathbf{W}_{hh} + \mathbf{b}_h)$$

The $\odot$ symbol represents the element-wise (Hadamard) product.

GRU Architecture: Hidden State

Step 2: The Final Hidden State ($\mathbf{H}_t$)

The final hidden state is a linear interpolation between the previous hidden state $\mathbf{H}_{t-1}$ and the new candidate hidden state $\tilde{\mathbf{H}}_t$. The update gate $\mathbf{Z}_t$ determines the mixing ratio.

$$\mathbf{H}_t = \mathbf{Z}_t \odot \mathbf{H}_{t-1} + (1 - \mathbf{Z}_t) \odot \tilde{\mathbf{H}}_t$$

This mechanism allows the GRU to either retain old information (if $\mathbf{Z}_t \approx 1$) or fully replace it with new information (if $\mathbf{Z}_t \approx 0$).

Modern Recurrent Neural Networks

Introduction to Bidirectional models and Encoder-Decoder architecture

Introduction to Bidirectional RNNs

The Limitation of Standard RNNs

- A standard RNN is unidirectional.

Introduction to Bidirectional RNNs

The Limitation of Standard RNNs

- A standard RNN is unidirectional.
- The hidden state at time t ($\mathbf{H}_t$) only captures information from past inputs ($\mathbf{X}_1, \ldots, \mathbf{X}_{t-1}$).

Introduction to Bidirectional RNNs

The Limitation of Standard RNNs

- A standard RNN is unidirectional.
- The hidden state at time t ($\mathbf{H}_t$) only captures information from past inputs ($\mathbf{X}_1, \ldots, \mathbf{X}_{t-1}$).
- This is a major drawback for tasks where future context is crucial.

Introduction to Bidirectional RNNs

The Limitation of Standard RNNs

- A standard RNN is unidirectional.
- The hidden state at time t ($\mathbf{H}_t$) only captures information from past inputs ($\mathbf{X}_1, \ldots, \mathbf{X}_{t-1}$).
- This is a major drawback for tasks where future context is crucial.
- **Example:** In the sentence "`The man who ___ movies is a critic`", we need the word "movies" to predict "reviews". A forward-only RNN would struggle to guess this from "The man who".

Introduction to Bidirectional RNN

The Solution: Look Both Ways

A **Bidirectional RNN (Bi-RNN)** solves this by processing the sequence in two directions simultaneously.

Introduction to Bidirectional RNN

The Solution: Look Both Ways

A **Bidirectional RNN (Bi-RNN)** solves this by processing the sequence in two directions simultaneously.

- It consists of two independent RNNs: one that processes the sequence from left-to-right (a **forward layer**) and another that processes it from right-to-left (a **backward layer**).

Introduction to Bidirectional RNN

The Solution: Look Both Ways

A **Bidirectional RNN (Bi-RNN)** solves this by processing the sequence in two directions simultaneously.

- It consists of two independent RNNs: one that processes the sequence from left-to-right (a **forward layer**) and another that processes it from right-to-left (a **backward layer**).
- For any time step t, the final hidden state is formed by concatenating the hidden states from both the forward and backward layers.

Training of Bidirectional RNNs

Training a Bi-RNN

- A **forward pass** from $t = 1$ to T is done to compute the forward hidden states ($\overrightarrow{\mathbf{H}}_t$).

Training of Bidirectional RNNs

Training a Bi-RNN

- A **forward pass** from $t = 1$ to T is done to compute the forward hidden states ($\overrightarrow{\mathbf{H}}_t$).
- A **backward pass** from $t = T$ to 1 is done to compute the backward hidden states ($\overleftarrow{\mathbf{H}}_t$).

Training of Bidirectional RNNs

Training a Bi-RNN

- A **forward pass** from $t = 1$ to T is done to compute the forward hidden states ($\overrightarrow{\mathbf{H}}_t$).
- A **backward pass** from $t = T$ to 1 is done to compute the backward hidden states ($\overleftarrow{\mathbf{H}}_t$).
- The output layer then uses the concatenated state $\mathbf{H}_t = [\overrightarrow{\mathbf{H}}_t, \overleftarrow{\mathbf{H}}_t]$ to make predictions.

Training of Bidirectional RNNs

Training a Bi-RNN

- A **forward pass** from $t = 1$ to T is done to compute the forward hidden states ($\overrightarrow{\mathbf{H}}_t$).
- A **backward pass** from $t = T$ to 1 is done to compute the backward hidden states ($\overleftarrow{\mathbf{H}}_t$).
- The output layer then uses the concatenated state $\mathbf{H}_t = [\overrightarrow{\mathbf{H}}_t, \overleftarrow{\mathbf{H}}_t]$ to make predictions.
- At Last output layer,

$$\mathbf{O}_t = \mathbf{H}_t \mathbf{W}_{hq} + \mathbf{b}_q$$

is computed and loss is backpropagated

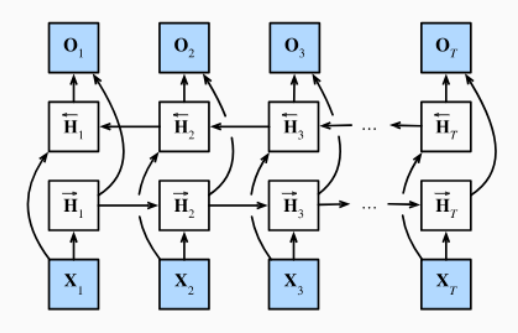


Figure: Bidirectional RNN. (source: D2L.ai)

Bidirectional RNNs: Illustrative Walkthrough

The Task: Predicting a Missing Word

Our goal is to understand how a Bidirectional RNN predicts the missing word in the sentence below. To make an accurate guess, the model needs to understand the context from both before and after the blank space.

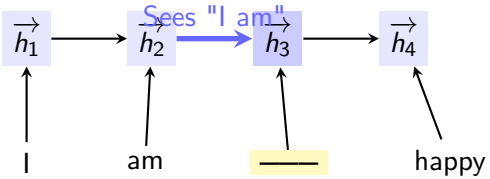
I am ——— happy

Let's walk through the process step-by-step.

Step 1: The Forward Pass (Past Context)

Reading from Left to Right

First, a standard RNN processes the sequence from the beginning. When it reaches the blank space at time step 3, its hidden state ($\overrightarrow{h_3}$) has only processed the past context: "I am".



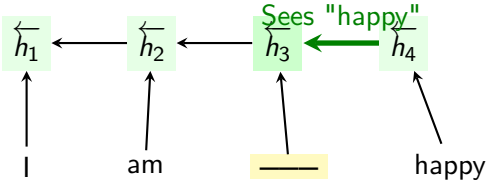
Limitation

This information alone is not enough to make a good prediction.

Step 2: The Backward Pass (Future Context)

Reading from Right to Left

Next, a second, independent RNN processes the sequence in reverse. Its hidden state at the blank space ($\overleftarrow{h}_3$) has only processed the future context: "happy".



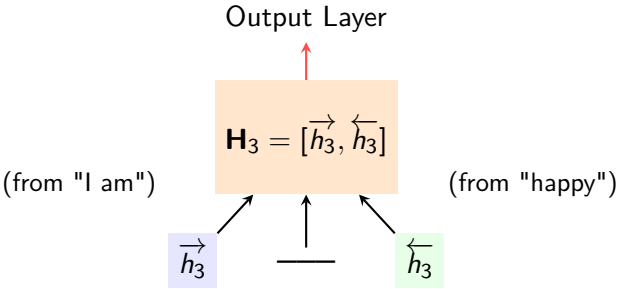
Limitation

This future context is helpful, but still incomplete on its own.

Step 3: Combining Context for Prediction

The Final Step: Concatenation

The Bidirectional RNN concatenates the hidden states from both the forward and backward passes. This combined state, $\mathbf{H}_3 = [\overrightarrow{h_3}, \overleftarrow{h_3}]$, now contains a rich representation of the full sentence context.



Encoder-Decoder Architectures

The Seq2Seq Challenge

Standard RNNs are only designed for tasks where the input and output lengths are the same. How do we handle tasks like machine translation or summarization, where the lengths can be completely different?

Encoder-Decoder Architectures

The Seq2Seq Challenge

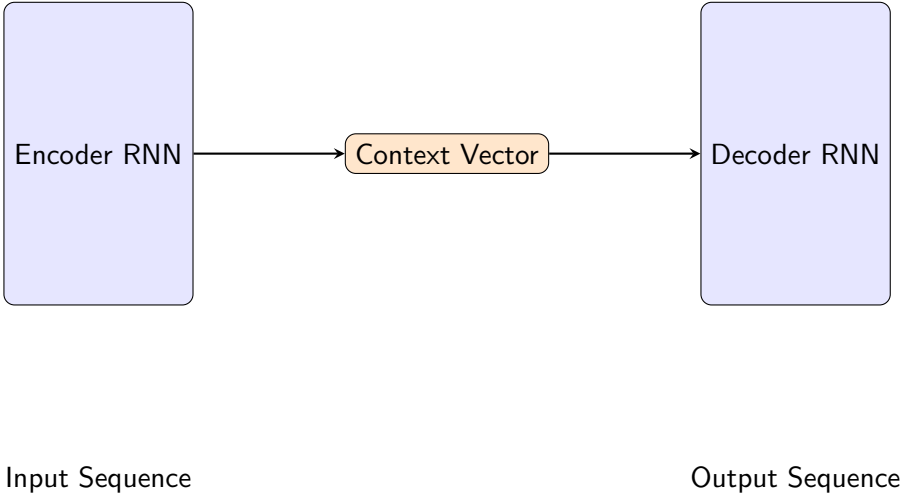
Standard RNNs are only designed for tasks where the input and output lengths are the same. How do we handle tasks like machine translation or summarization, where the lengths can be completely different?

The Encoder-Decoder Framework

This architecture, also known as a Sequence-to-Sequence (Seq2Seq) model, is designed specifically for this problem. It consists of two main RNN components:

- **The Encoder:** An RNN that reads the entire input sequence step-by-step. Its sole purpose is to compress the information of the whole sequence into a single fixed-size vector, often called the **context vector**.
- **The Decoder:** Another RNN that is initialized with the encoder's context vector. It generates the output sequence one token at a time, using the context and the previously generated token.

Encoder-Decoder Architectures



Training an Encoder-Decoder Architecture

- During training, instead of feeding the decoder's own prediction from the previous step as input for the current step, we use the actual ground-truth token from the target sequence. This technique is called **teacher forcing**.

Training an Encoder-Decoder Architecture

- During training, instead of feeding the decoder's own prediction from the previous step as input for the current step, we use the actual ground-truth token from the target sequence. This technique is called **teacher forcing**.
- **Benefit:** This approach stabilizes training and helps the model learn the sequence structure more efficiently, as it always receives correct input.

Training an Encoder-Decoder Architecture

- During training, instead of feeding the decoder's own prediction from the previous step as input for the current step, we use the actual ground-truth token from the target sequence. This technique is called **teacher forcing**.
- **Benefit:** This approach stabilizes training and helps the model learn the sequence structure more efficiently, as it always receives correct input.
- At inference time, however, the model must use its own previously generated tokens as input, since the ground truth is not available.

Training an Encoder-Decoder Architecture

- During training, instead of feeding the decoder's own prediction from the previous step as input for the current step, we use the actual ground-truth token from the target sequence. This technique is called **teacher forcing**.
- **Benefit:** This approach stabilizes training and helps the model learn the sequence structure more efficiently, as it always receives correct input.
- At inference time, however, the model must use its own previously generated tokens as input, since the ground truth is not available.
- The use of Context Vector in a decoder follows a design choice, but in most cases it is concatenated with the input sequences to the decoder model.

Loss Function and Optimization

- The encoder and decoder models are trained jointly for a sequence to sequence task.

Loss Function and Optimization

- The encoder and decoder models are trained jointly for a sequence to sequence task.
- The model is typically trained using a **cross-entropy loss**.

Loss Function and Optimization

- The encoder and decoder models are trained jointly for a sequence to sequence task.
- The model is typically trained using a **cross-entropy loss**.
- The loss is calculated at each timestep of the decoder's output by comparing the predicted probability distribution over the vocabulary with the one-hot encoded ground-truth token.

Loss Function and Optimization

- The encoder and decoder models are trained jointly for a sequence to sequence task.
- The model is typically trained using a **cross-entropy loss**.
- The loss is calculated at each timestep of the decoder's output by comparing the predicted probability distribution over the vocabulary with the one-hot encoded ground-truth token.
- **Masking:** When working with batches of sequences that have different lengths, padding is used to make them uniform. The loss function uses a mask to ignore these padding tokens during calculation, ensuring they do not contribute to the gradient updates.

Evaluating Predicted Sequences: The BLEU Score

- **BLEU** (Bilingual Evaluation Understudy) is a standard metric for evaluating the quality of a predicted sequence by comparing it to a target (ground truth) sequence.

Evaluating Predicted Sequences: The BLEU Score

- **BLEU** (Bilingual Evaluation Understudy) is a standard metric for evaluating the quality of a predicted sequence by comparing it to a target (ground truth) sequence.
- It is based on the precision of matching **n-grams** (contiguous sequences of n items) between the two sequences.

Evaluating Predicted Sequences: The BLEU Score

- **BLEU** (Bilingual Evaluation Understudy) is a standard metric for evaluating the quality of a predicted sequence by comparing it to a target (ground truth) sequence.
- It is based on the precision of matching **n-grams** (contiguous sequences of n items) between the two sequences.
- Let p_n be the precision of n-grams, i.e., the ratio of the number of matched n-grams in the predicted and target sequences to the number of n-grams in the predicted sequence.

BLEU Formula:

$$\text{BLEU} = \exp\left(\min\left(0, 1 - \frac{\text{len}_{\text{label}}}{\text{len}_{\text{pred}}}\right)\right) \cdot \left(\prod_{n=1}^{k} p_n\right)^{1/k}$$

Evaluating Predicted Sequences: The BLEU Score

- **BLEU** (Bilingual Evaluation Understudy) is a standard metric for evaluating the quality of a predicted sequence by comparing it to a target (ground truth) sequence.
- It is based on the precision of matching **n-grams** (contiguous sequences of n items) between the two sequences.
- Let p_n be the precision of n-grams, i.e., the ratio of the number of matched n-grams in the predicted and target sequences to the number of n-grams in the predicted sequence.

BLEU Formula:

$$\text{BLEU} = \exp\left(\min\left(0, 1 - \frac{\text{len}_{\text{label}}}{\text{len}_{\text{pred}}}\right)\right) \cdot \left(\prod_{n=1}^{k} p_n\right)^{1/k}$$

where:

- k is the longest n-gram to consider.

Evaluating Predicted Sequences: The BLEU Score

- **BLEU** (Bilingual Evaluation Understudy) is a standard metric for evaluating the quality of a predicted sequence by comparing it to a target (ground truth) sequence.
- It is based on the precision of matching **n-grams** (contiguous sequences of n items) between the two sequences.
- Let p_n be the precision of n-grams, i.e., the ratio of the number of matched n-grams in the predicted and target sequences to the number of n-grams in the predicted sequence.

BLEU Formula:

$$\text{BLEU} = \exp\left(\min\left(0, 1 - \frac{\text{len}_{\text{label}}}{\text{len}_{\text{pred}}}\right)\right) \cdot \left(\prod_{n=1}^{k} p_n\right)^{1/k}$$

where:

- k is the longest n-gram to consider.
- $\text{len}_{\text{label}}$ and len_{pred} denote the number of tokens in the target sequence and the predicted sequence, respectively.

Evaluating Predicted Sequences: The BLEU Score

- **BLEU** (Bilingual Evaluation Understudy) is a standard metric for evaluating the quality of a predicted sequence by comparing it to a target (ground truth) sequence.
- It is based on the precision of matching **n-grams** (contiguous sequences of n items) between the two sequences.
- Let p_n be the precision of n-grams, i.e., the ratio of the number of matched n-grams in the predicted and target sequences to the number of n-grams in the predicted sequence.

BLEU Formula:

$$\text{BLEU} = \exp\left(\min\left(0, 1 - \frac{\text{len}_{\text{label}}}{\text{len}_{\text{pred}}}\right)\right) \cdot \left(\prod_{n=1}^{k} p_n\right)^{1/k}$$

where:

- k is the longest n-gram to consider.
- $\text{len}_{\text{label}}$ and len_{pred} denote the number of tokens in the target sequence and the predicted sequence, respectively.
- If the predicted sequence is identical to the target, the BLEU score is 1.

BLEU Score: A Numerical Example

Let's consider an example with $k = 2$ (we evaluate unigrams and bigrams).

- **Target (Label) Sequence**: "A B C D E F"
- **Predicted Sequence**: "A B"

1. Calculate n-gram Precision (p_n):

- **Unigram Precision (p_1):**
  - Predicted unigrams: {'A', 'B'}
  - All are found in the target sequence.
  - $p_1 = \frac{\text{matched}}{\text{total}} = \frac{2}{2} = 1.0$
- **Bigram Precision (p_2):**
  - Predicted bigrams: {'A B'}
  - The bigram is found in the target sequence.
  - $p_2 = \frac{\text{matched}}{\text{total}} = \frac{1}{1} = 1.0$

Without a penalty, the score would be $(1.0 \times 1.0)^{1/2} = 1.0$, which is misleadingly perfect.

Applying the penalty term in BLEU

2. Apply the Brevity Penalty (BP):

- The predicted sequence is shorter than the target ($\text{len}_{\text{pred}} = 2$, $\text{len}_{\text{label}} = 6$).
- We must apply the penalty:

$$BP = \exp\left(1 - \frac{\text{len}_{\text{label}}}{\text{len}_{\text{pred}}}\right) = \exp\left(1 - \frac{6}{2}\right) = \exp(-2) \approx 0.135$$

Applying the penalty term in BLEU

2. Apply the Brevity Penalty (BP):

- The predicted sequence is shorter than the target ($\text{len}_{\text{pred}} = 2$, $\text{len}_{\text{label}} = 6$).
- We must apply the penalty:

$$BP = \exp\left(1 - \frac{\text{len}_{\text{label}}}{\text{len}_{\text{pred}}}\right) = \exp\left(1 - \frac{6}{2}\right) = \exp(-2) \approx 0.135$$

3. Final BLEU Score:

$$\text{BLEU} = BP \cdot (p_1 \cdot p_2)^{1/2} \approx 0.135 \times (1.0 \times 1.0)^{1/2} = 0.135$$

Conclusion: The brevity penalty correctly lowers the score from a perfect 1.0 to 0.135, reflecting that while the precision is high, the prediction is far too short.

Applications of Encoder-Decoder Architectures

Where Seq2Seq Models Shine

The Encoder-Decoder framework is the foundation for many state-of-the-art NLP and sequence-based tasks where the input and output lengths differ.

- **Machine Translation** (e.g., translating a 10-word English sentence to a 12-word French sentence).

Applications of Encoder-Decoder Architectures

Where Seq2Seq Models Shine

The Encoder-Decoder framework is the foundation for many state-of-the-art NLP and sequence-based tasks where the input and output lengths differ.

- **Machine Translation** (e.g., translating a 10-word English sentence to a 12-word French sentence).
- **Text Summarization** (e.g., reading a 500-word article and generating a 50-word summary).

Applications of Encoder-Decoder Architectures

Where Seq2Seq Models Shine

The Encoder-Decoder framework is the foundation for many state-of-the-art NLP and sequence-based tasks where the input and output lengths differ.

- **Machine Translation** (e.g., translating a 10-word English sentence to a 12-word French sentence).
- **Text Summarization** (e.g., reading a 500-word article and generating a 50-word summary).
- **Question Answering** (e.g., reading a context paragraph and generating a concise answer).

The Challenge of Sequence Prediction

From Greedy to Optimal Search

- When generating a sequence (e.g., a sentence), our goal is to find the sequence with the highest conditional probability.

The Challenge of Sequence Prediction

From Greedy to Optimal Search

- When generating a sequence (e.g., a sentence), our goal is to find the sequence with the highest conditional probability.
- **Greedy Search**: At each step, we choose the single most likely token.
  - **Pro**: It's fast and computationally cheap.
  - **Con**: It is not optimal. The sequence of most-likely tokens is not necessarily the most-likely sequence overall.

The Challenge of Sequence Prediction

From Greedy to Optimal Search

- When generating a sequence (e.g., a sentence), our goal is to find the sequence with the highest conditional probability.
- **Greedy Search**: At each step, we choose the single most likely token.
  - **Pro**: It's fast and computationally cheap.
  - **Con**: It is not optimal. The sequence of most-likely tokens is not necessarily the most-likely sequence overall.
- **Exhaustive Search**: We could calculate the probability of every single possible sequence.
  - **Pro**: It guarantees finding the optimal sequence.
  - **Con**: The cost is very high, which is computationally infeasible for any practical application.

Beam Search: A Practical Compromise

- Beam search is a search algorithm that explores the most promising options in a limited way at each step.

Beam Search: A Practical Compromise

- Beam search is a search algorithm that explores the most promising options in a limited way at each step.
- It strikes a balance between the speed of greedy search and the optimality of exhaustive search.

Beam Search: A Practical Compromise

- Beam search is a search algorithm that explores the most promising options in a limited way at each step.
- It strikes a balance between the speed of greedy search and the optimality of exhaustive search.
- The core idea is controlled by a hyperparameter called the **beam size**, k.

Beam Search: A Practical Compromise

- Beam search is a search algorithm that explores the most promising options in a limited way at each step.
- It strikes a balance between the speed of greedy search and the optimality of exhaustive search.
- The core idea is controlled by a hyperparameter called the **beam size**, k.

How it works:

1. **Step 1**: Instead of choosing only the single best token, we select the top k most probable tokens to start our sequences.

Beam Search: A Practical Compromise

- Beam search is a search algorithm that explores the most promising options in a limited way at each step.
- It strikes a balance between the speed of greedy search and the optimality of exhaustive search.
- The core idea is controlled by a hyperparameter called the **beam size**, k.

How it works:

1. **Step 1**: Instead of choosing only the single best token, we select the top k most probable tokens to start our sequences.
2. **Subsequent Steps**: For each of the k candidate sequences from the previous step, we expand it with every possible next token. This gives us $k \times |\mathcal{Y}|$ new candidate sequences.

Beam Search: A Practical Compromise

- Beam search is a search algorithm that explores the most promising options in a limited way at each step.
- It strikes a balance between the speed of greedy search and the optimality of exhaustive search.
- The core idea is controlled by a hyperparameter called the **beam size**, k.

How it works:

1. **Step 1**: Instead of choosing only the single best token, we select the top k most probable tokens to start our sequences.
2. **Subsequent Steps**: For each of the k candidate sequences from the previous step, we expand it with every possible next token. This gives us $k \times |\mathcal{Y}|$ new candidate sequences.
3. From this large set, we again select the top k sequences with the highest overall probability.

Beam Search: A Practical Compromise

- Beam search is a search algorithm that explores the most promising options in a limited way at each step.
- It strikes a balance between the speed of greedy search and the optimality of exhaustive search.
- The core idea is controlled by a hyperparameter called the **beam size**, k.

How it works:

1. **Step 1**: Instead of choosing only the single best token, we select the top k most probable tokens to start our sequences.
2. **Subsequent Steps**: For each of the k candidate sequences from the previous step, we expand it with every possible next token. This gives us $k \times |\mathcal{Y}|$ new candidate sequences.
3. From this large set, we again select the top k sequences with the highest overall probability.
4. This process is repeated until an end-of-sequence token is generated or a maximum length is reached.

Beam Search: An Illustration

Beam Size, k=2

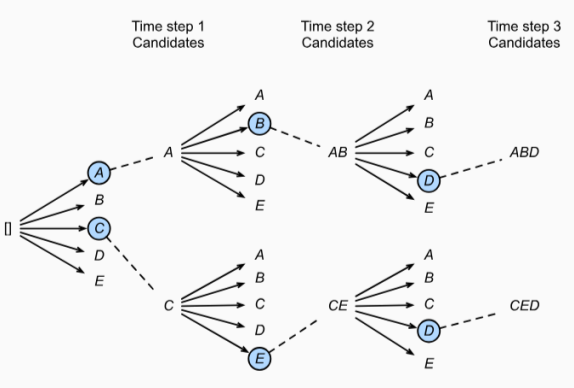


Figure: In the above example candidate solutions are: {A}, {C}, {A,B}, {C,E}, {A,B,D}, {C,E,D}(source: d2l.ai)

Final Selection and Scoring

- After the search is complete, we have a set of final candidate sequences.

Final Selection and Scoring

- After the search is complete, we have a set of final candidate sequences.
- We choose the candidate sequence that maximizes the joint probability distribution.

Final Selection and Scoring

- After the search is complete, we have a set of final candidate sequences.
- We choose the candidate sequence that maximizes the joint probability distribution.
- A simple product of probabilities is a way, but it is biased towards shorter sequences.

Final Selection and Scoring

- After the search is complete, we have a set of final candidate sequences.
- We choose the candidate sequence that maximizes the joint probability distribution.
- A simple product of probabilities is a way, but it is biased towards shorter sequences.
- To counteract this, the final score is often normalized by the sequence length:

$$\text{Score} = \frac{1}{L^\alpha} \log P(y_1, \ldots, y_L | c) = \frac{1}{L^\alpha} \sum_{t'=1}^{L} \log P(y_{t'} | y_1, \ldots, y_{t'-1}, c)$$

Final Selection and Scoring

- After the search is complete, we have a set of final candidate sequences.
- We choose the candidate sequence that maximizes the joint probability distribution.
- A simple product of probabilities is a way, but it is biased towards shorter sequences.
- To counteract this, the final score is often normalized by the sequence length:

$$\text{Score} = \frac{1}{L^\alpha} \log P(y_1, \ldots, y_L | c) = \frac{1}{L^\alpha} \sum_{t'=1}^{L} \log P(y_{t'} | y_1, \ldots, y_{t'-1}, c)$$

where:

- L is the length of the candidate sequence.
- α is a hyperparameter (e.g., 0.75) that controls the strength of the length penalty. The term L^α in the denominator penalizes long sequences.

Final Selection and Scoring

- After the search is complete, we have a set of final candidate sequences.
- We choose the candidate sequence that maximizes the joint probability distribution.
- A simple product of probabilities is a way, but it is biased towards shorter sequences.
- To counteract this, the final score is often normalized by the sequence length:

$$\text{Score} = \frac{1}{L^\alpha} \log P(y_1, \ldots, y_L | c) = \frac{1}{L^\alpha} \sum_{t'=1}^{L} \log P(y_{t'} | y_1, \ldots, y_{t'-1}, c)$$

where:

- L is the length of the candidate sequence.
- α is a hyperparameter (e.g., 0.75) that controls the strength of the length penalty. The term L^α in the denominator penalizes long sequences.

Note

Greedy search is a special case of beam search arising when the beam size is set to 1.